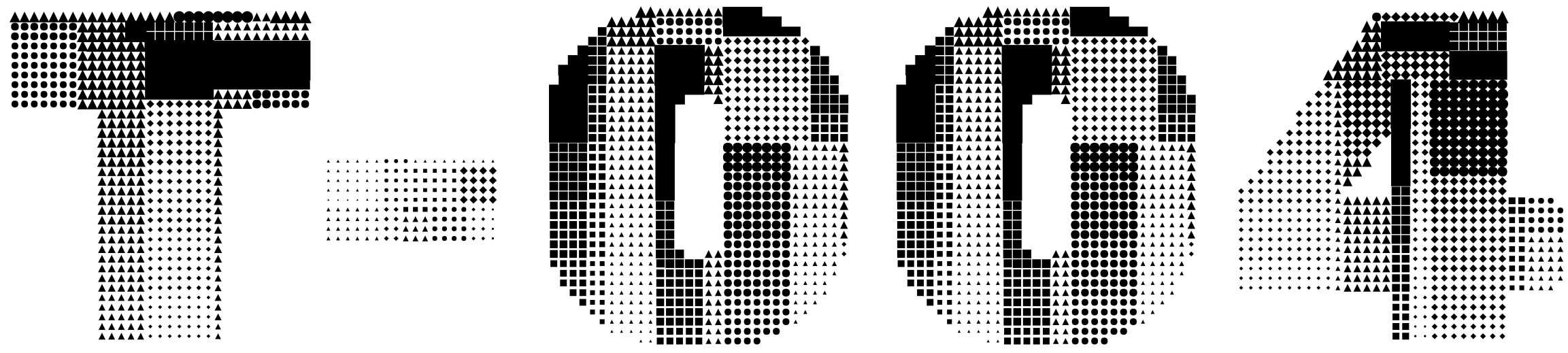
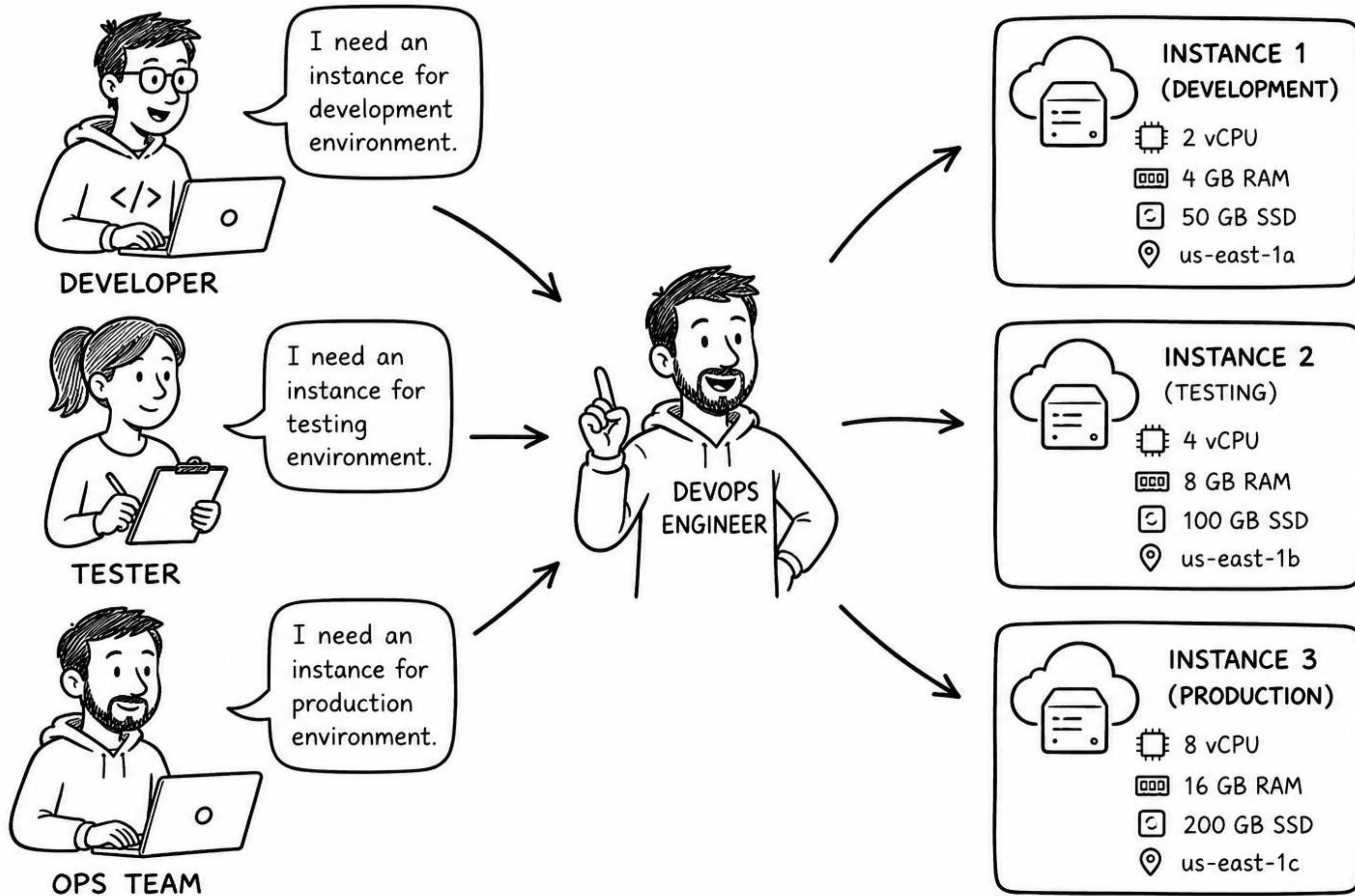
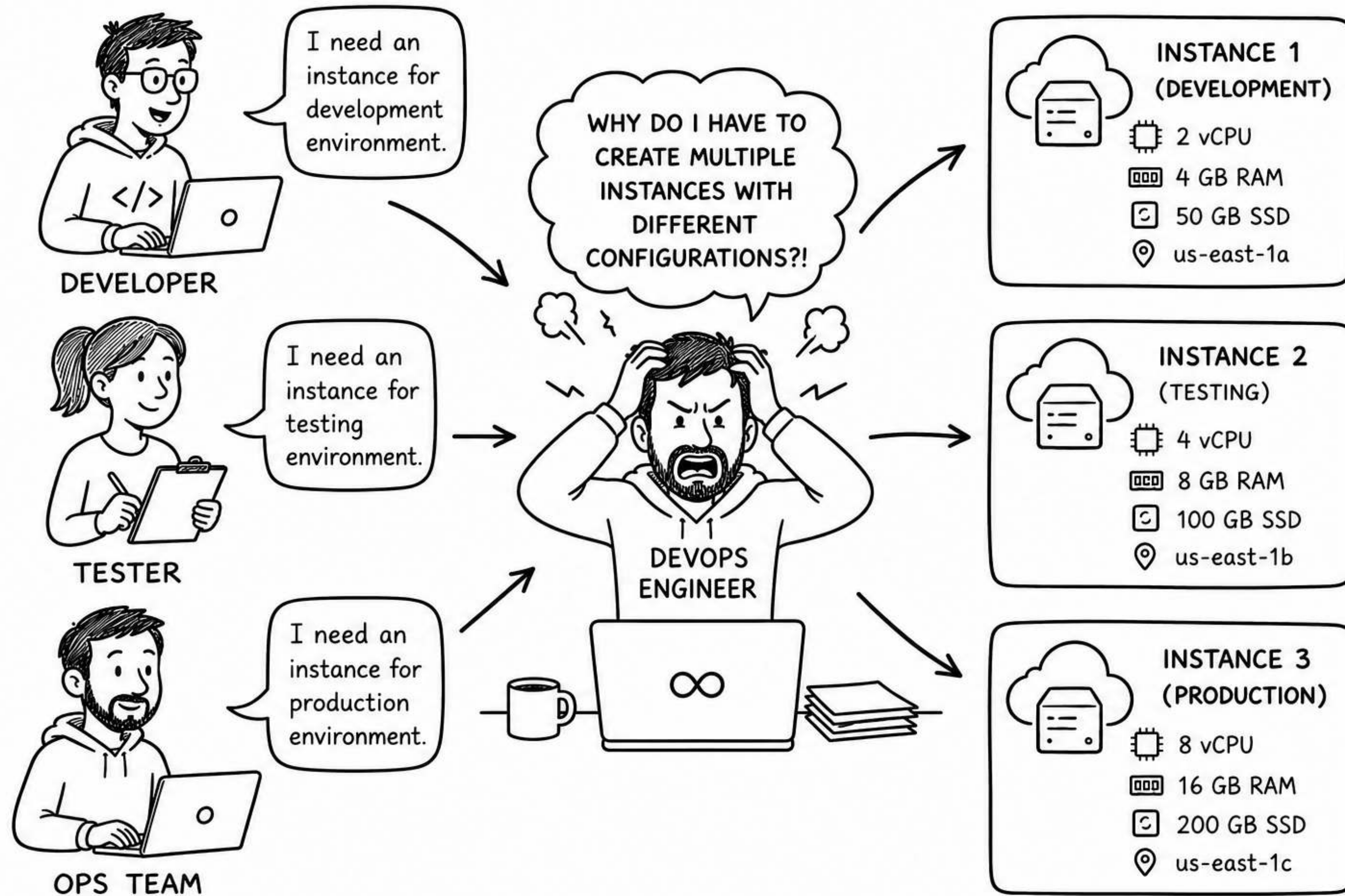
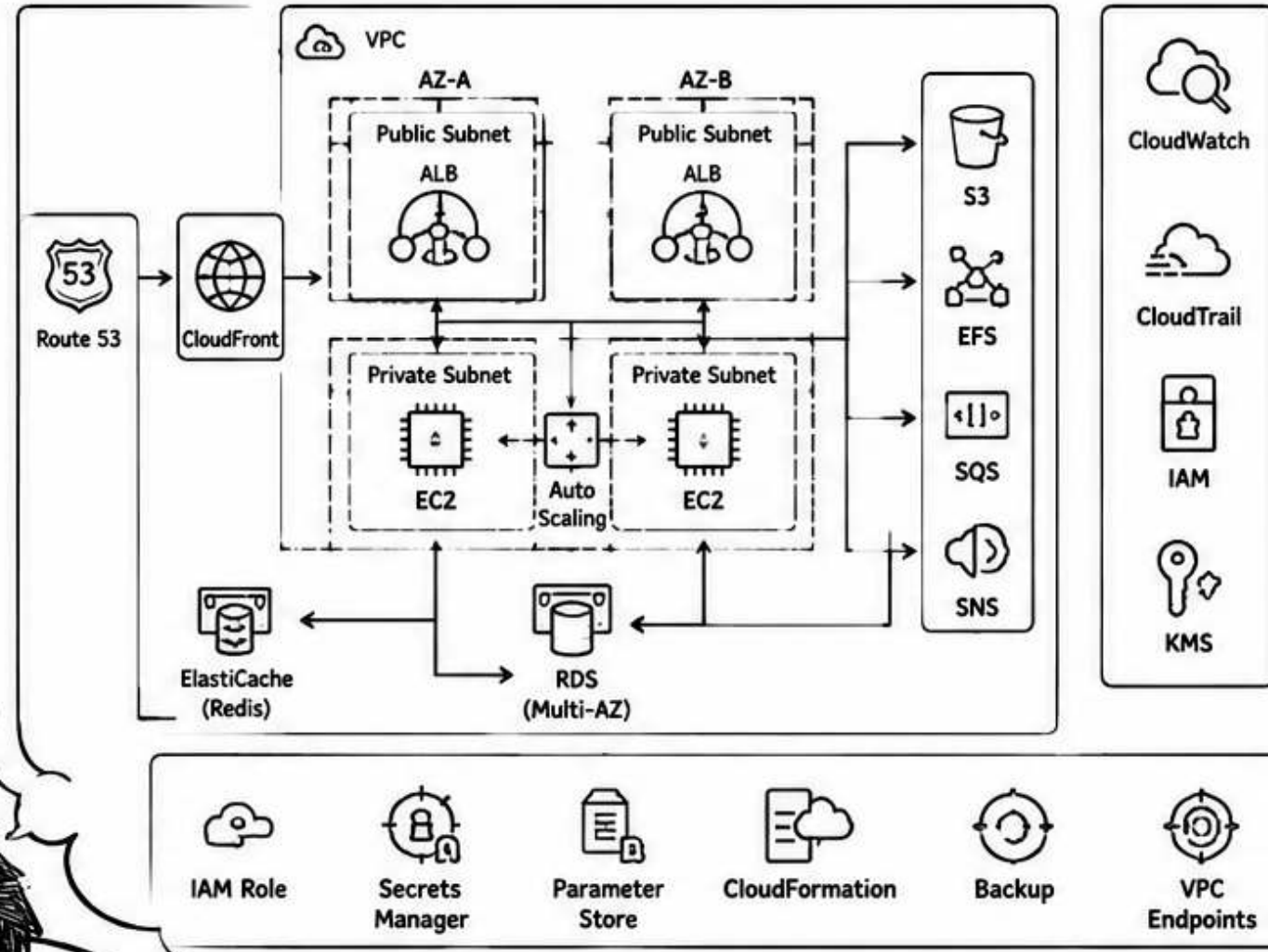
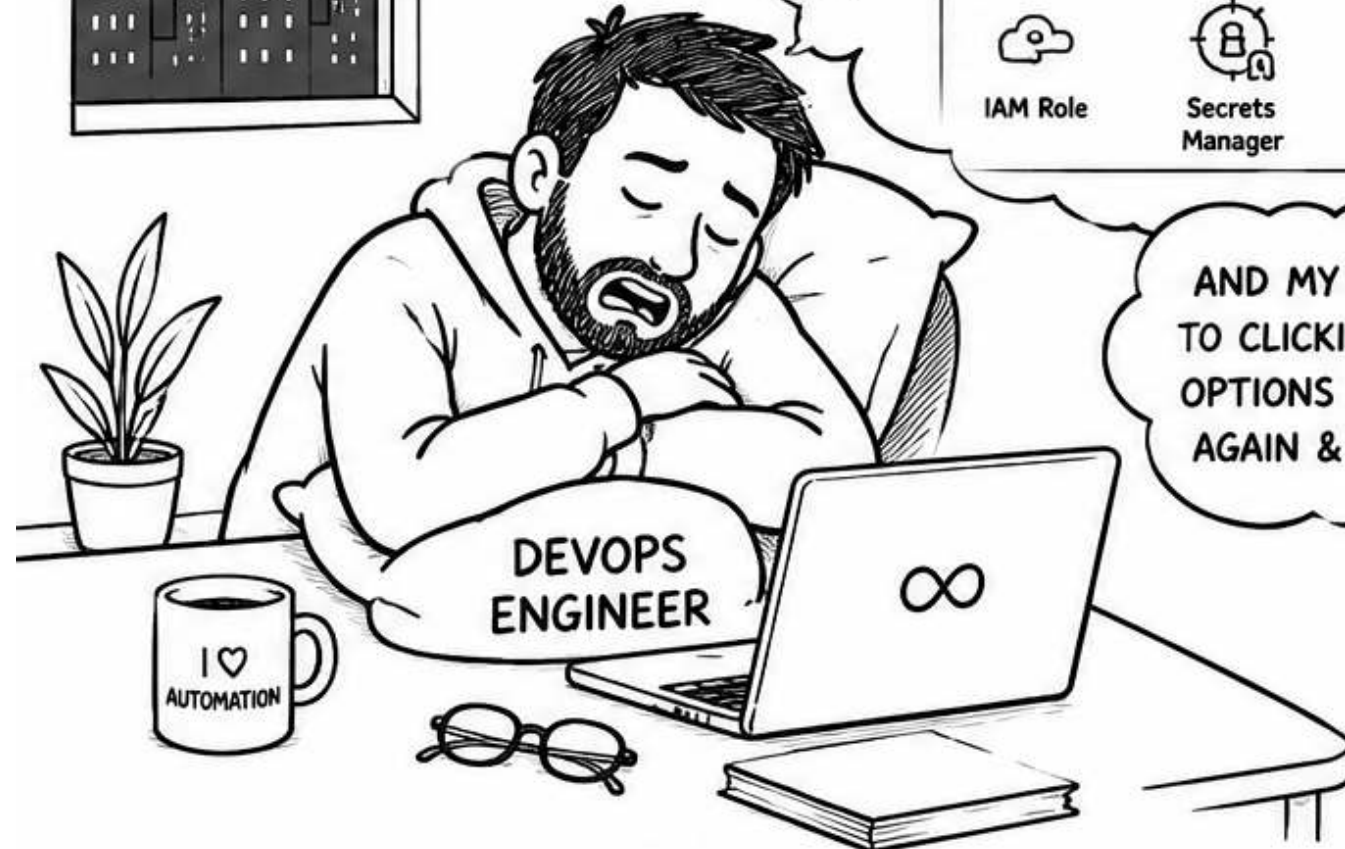








TODAY I CREATED
3 INSTANCES, BUT
HOW ABOUT THESE
KIND OF LARGE
ARCHITECTURES?



IT'S GOING TO BE
COMPLETE MANUAL!

Create VPC

CIDR Block

Tenancy

Create

CLICK!

Create Subnet

VPC

CIDR Block

AZ

Create

CLICK!

Launch Instance

AMI

Instance Type

Launch

CLICK!

AND MY ENTIRE LIFE IS
TO CLICKING THOSE MANUAL
OPTIONS TO CREATE INFRA
AGAIN & AGAIN...

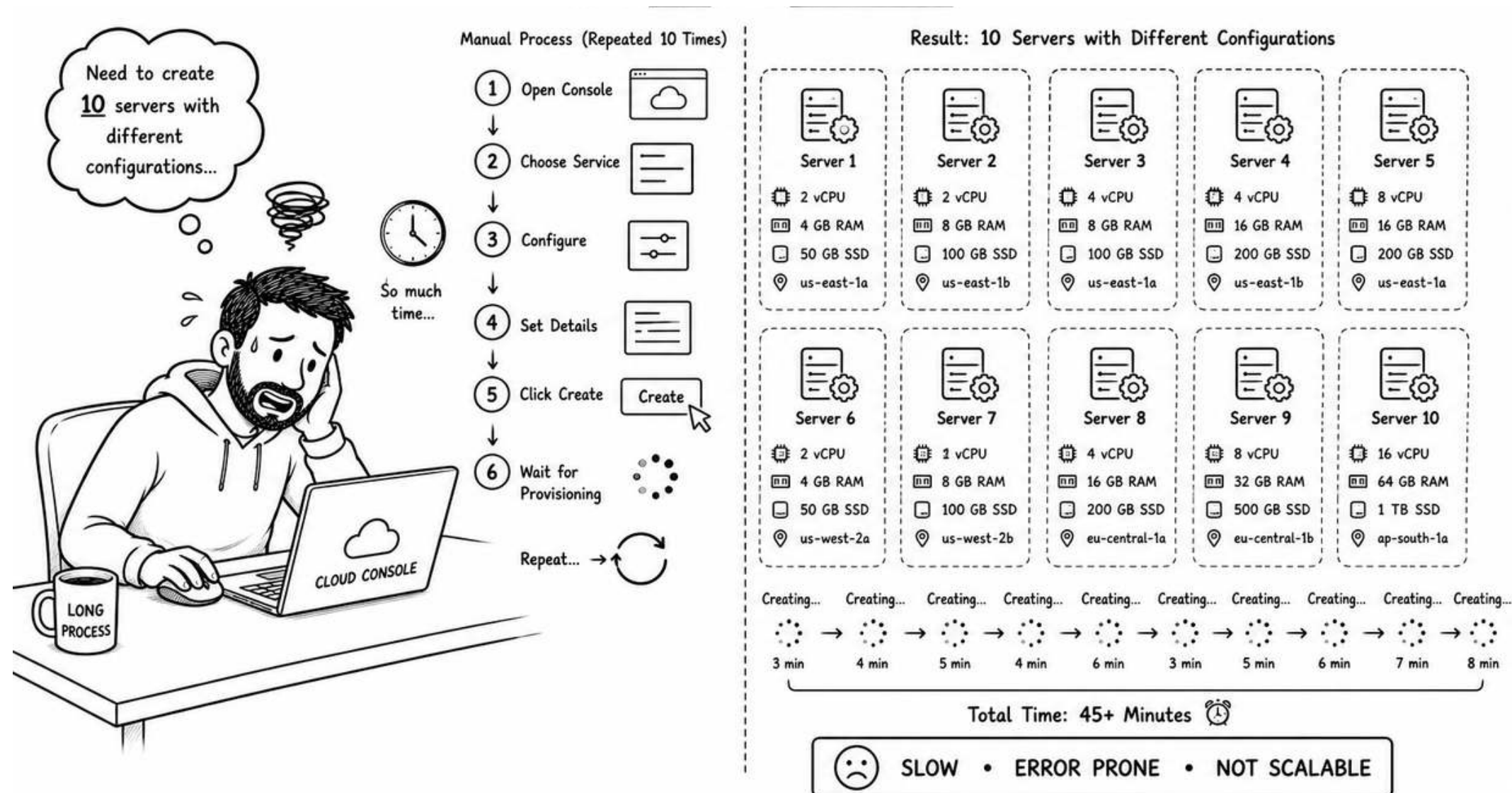
MY LIFE =
CLICKING





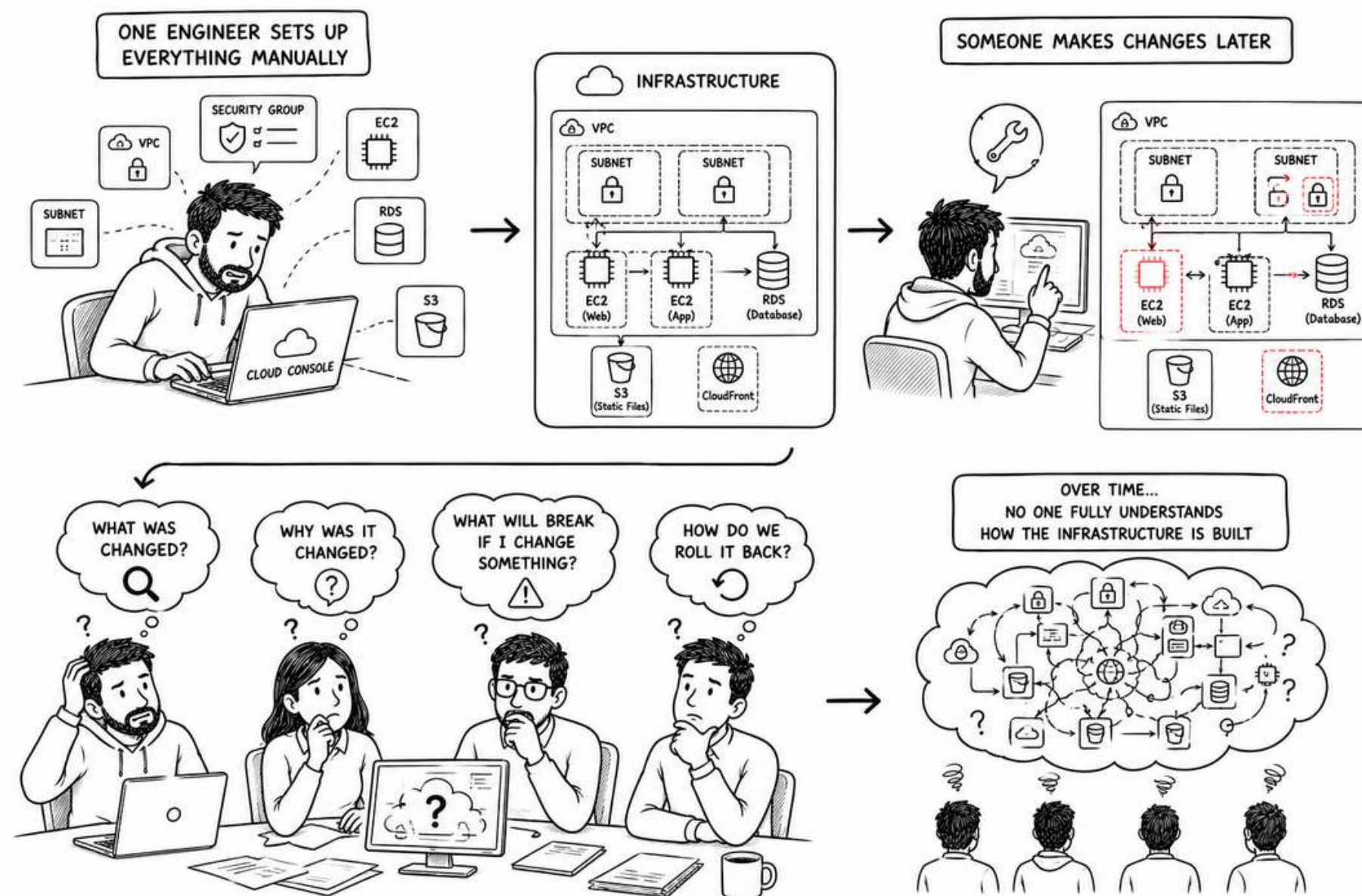
Some Case Studies Without IAC

1. **Slow setup:** If a team needs 10 servers with different configurations, engineers must manually click through the cloud console, wait for provisioning, and repeat the process multiple times. This is slow and not scalable.



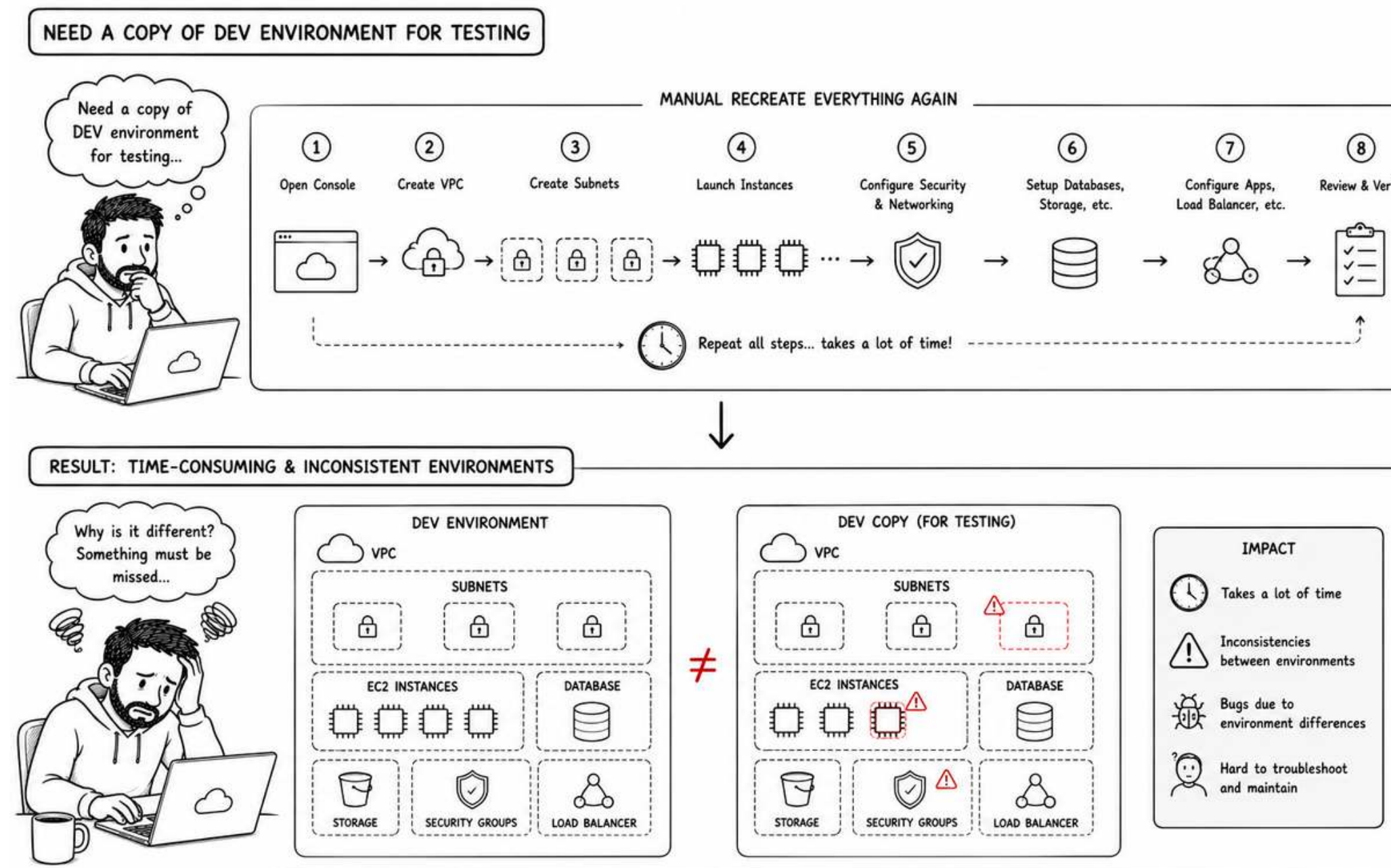
Some Case Studies Without IAC

2. Lack of visibility & control: Often, a single engineer sets up the entire infrastructure manually. If someone later changes something, it becomes difficult to track what was changed, why it was changed, or how to roll it back. Over time, no one fully understands how the infrastructure is actually built



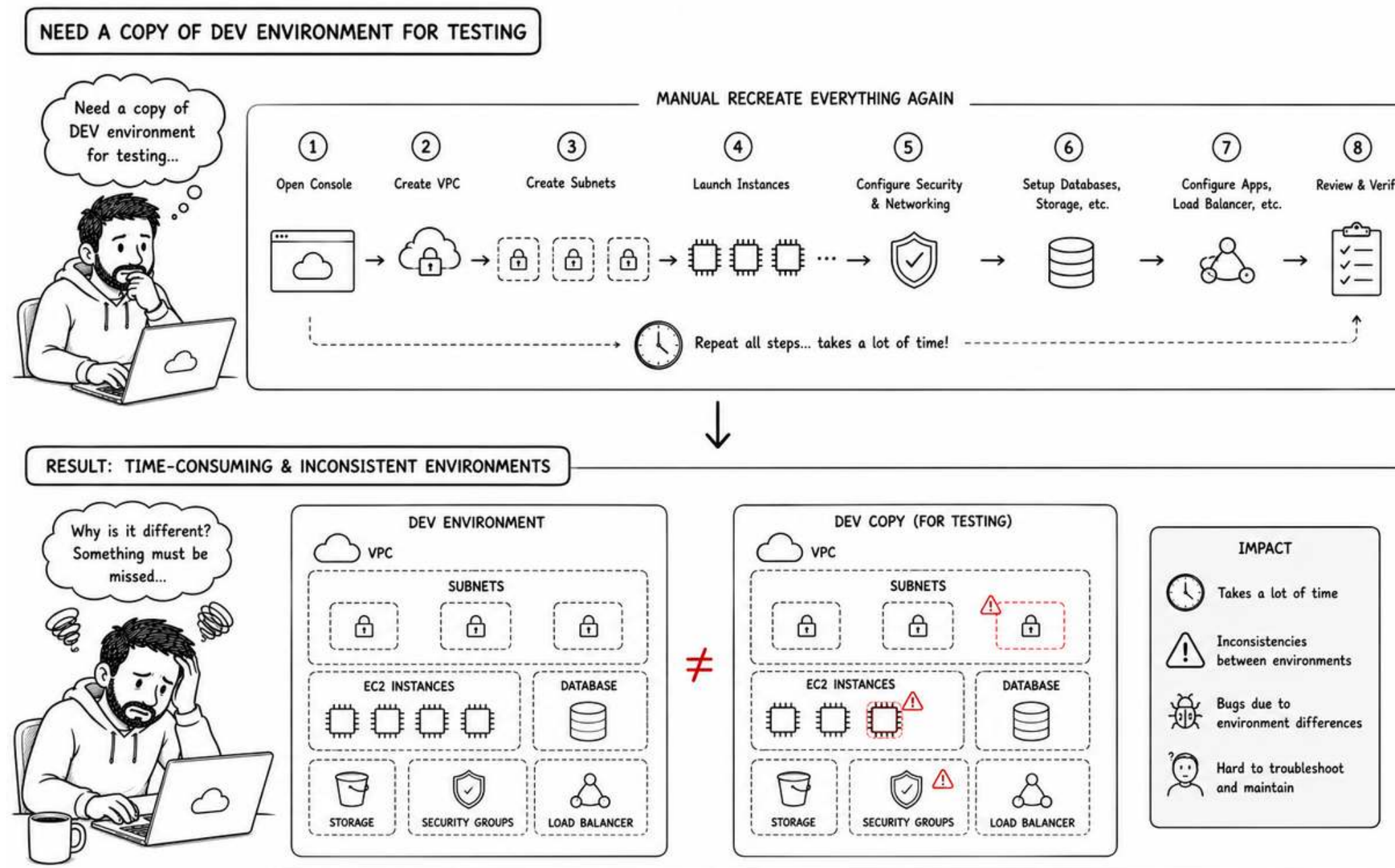
Some Case Studies Without IAC

3. Difficult to recreate environments: If we need a copy of a development environment for testing, we must manually recreate the entire infrastructure again. This is time-consuming and may lead to inconsistencies between environments.



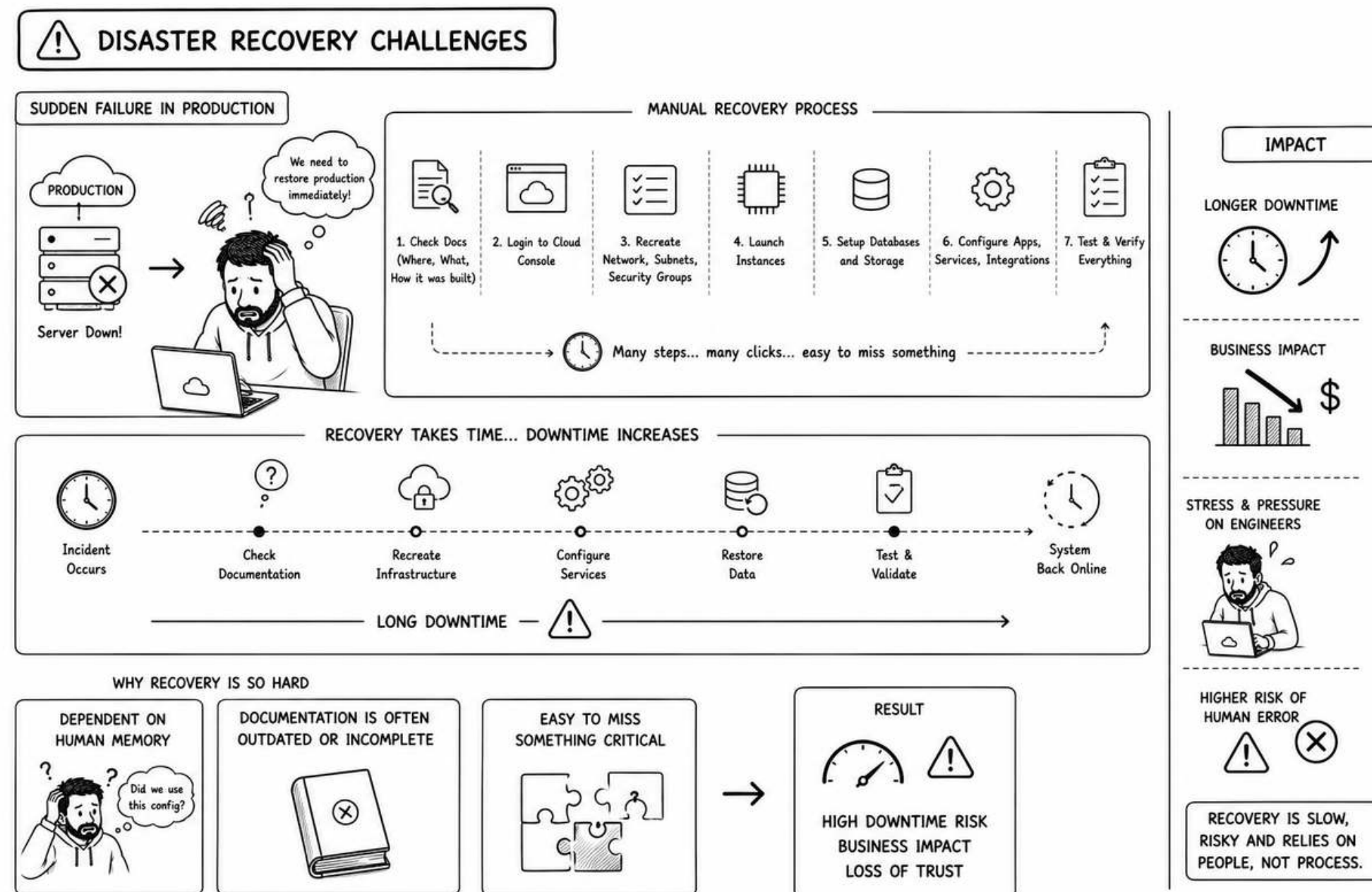
Some Case Studies Without IAC

4. **Manual errors:** During manual server creation, incorrect configuration (wrong OS version, instance type, security group, etc.) can cause the application to fail or behave unexpectedly in that environment.



Some Case Studies Without IAC

5. **Disaster recovery challenges:** If a production server crashes, rebuilding the system manually takes time and increases downtime risk. Recovery is slow and depends heavily on human memory and documentation.





IAC : Infrastructure as Code

IaC is the practice of managing infrastructure using machine-readable configuration files instead of manual processes.

Advantages:

1. **Speed & automation:** Write code once and deploy 1000's of servers in minutes.
2. **Consistency:** Every environment is created from the same code.
3. **Reusability:** Once you write infrastructure code, you can reuse it again and again for different environments and different projects
4. **Easy disaster recovery:** If something breaks, delete broken infra run IaC code again & rebuild everything quickly.
5. **Version control:** Infrastructure becomes code so we can track changes, see who changed what and rollback to previous version

MAIN IAC TOOLS (YOU SHOULD KNOW THESE)

① TERRAFORM (MOST POPULAR)



What it does:

Creates and manages infrastructure across any cloud

Think of it like:

☞ "Universal remote" control for AWS, Azure, GCP



Used for:

- creating servers (EC2, VMs)
- databases
- networking (VPC, subnets)
- load balancers



Why people love it:

- ✓ works with any cloud (multi-cloud)
- ✓ very widely used in companies
- ✓ tracks infrastructure state

② AWS CLOUDFORMATION



What it does:

IaC tool specifically for AWS

Think of it like:

☞ "AWS-only Terraform" (but native to AWS)



Used for:

- AWS resources only
- tightly integrated with AWS services



Strength:

- ✓ very reliable inside AWS ecosystem

Weakness:

- ✗ only works on AWS (not multi-cloud)

③ ANSIBLE



What it does:

Configures servers AFTER they are created

Think of it like:

☞ "Chef/maintenance worker inside the house"



Used for:

- installing software (Docker, Nginx)
- updating servers
- running scripts across many machines



Important:

- ! not mainly for creating infrastructure
- ✓ best for configuration automation

④ PULUMI



What it does:

Same as Terraform, but you write code in real programming languages

Think of it like:

☞ "Terraform, but using Python/JavaScript instead of YAML-like config"



Used for:

- cloud infrastructure
- modern DevOps teams who prefer coding style



Strength:

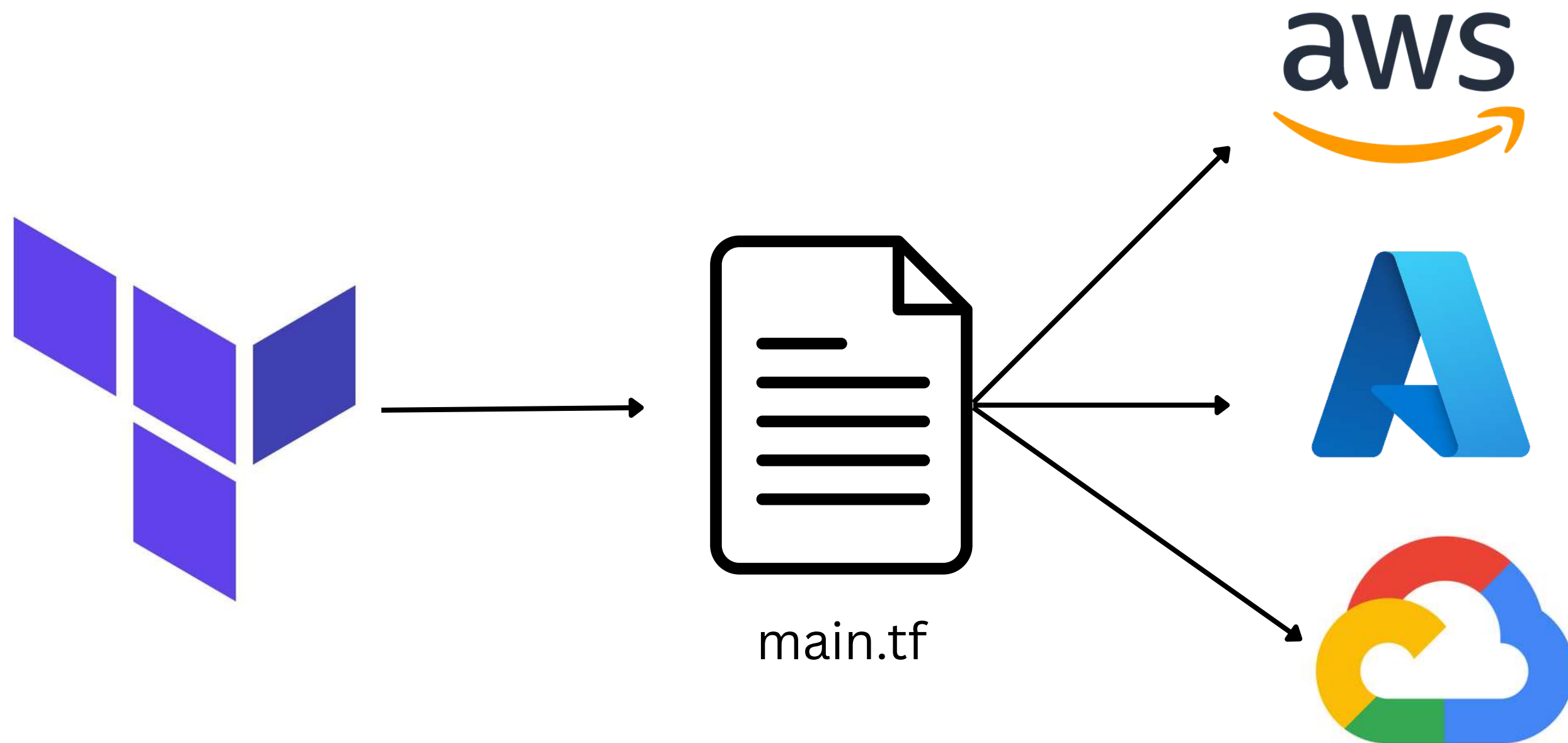
- ✓ use familiar languages (Python, TS, Go)

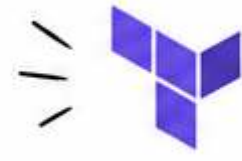
</>



What is Terraform?

Terraform is an Infrastructure as Code tool that automatically creates, updates, and manages infrastructure using configuration files on multi-cloud.





HOW TERRAFORM WORKS (VERY SIMPLE FLOW)

① YOU WRITE CODE

You describe infrastructure in files like:

- SERVERS
- DATABASES
- NETWORKS

Example idea:

"I want 2 servers in AWS"

```
main.tf
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "app" {
  count = 2
  ami   = "ami-123456"
  instance_type = "t2.micro"
}

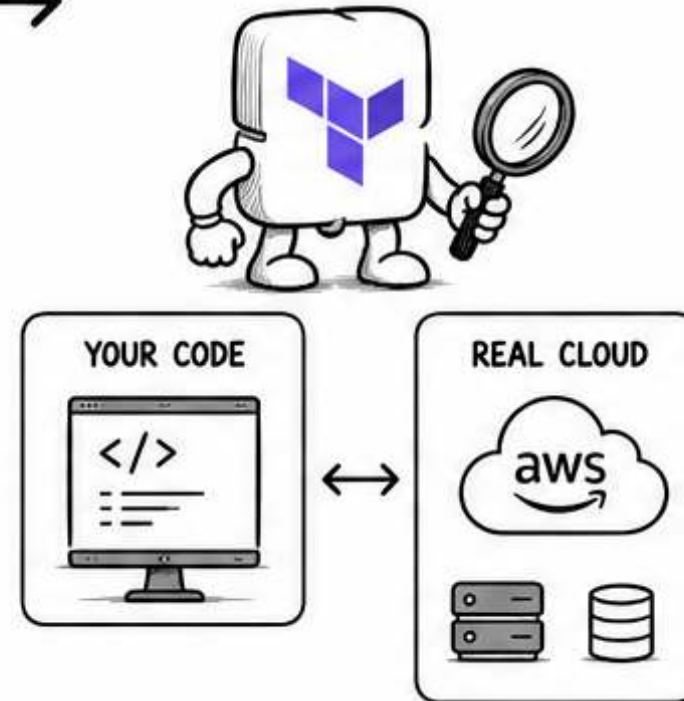
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

YOU DEFINE WHAT YOU WANT

② TERRAFORM READS IT

It compares:

- ✓ what you want (code)
- ✓ what already exists (real cloud)



TERRAFORM CHECKS THE DIFFERENCE

③ IT CREATES A PLAN

It shows:

I will create:

- ✓ 2 servers
- ✓ 1 network
- ✓ open port 80



YOU SEE EXACTLY WHAT WILL BE CHANGED

④ YOU APPLY IT

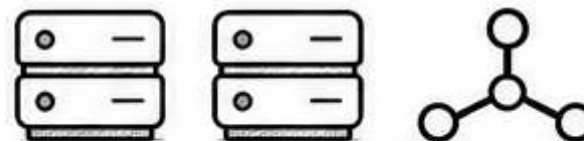
Terraform executes changes and builds everything automatically.

\$ terraform apply

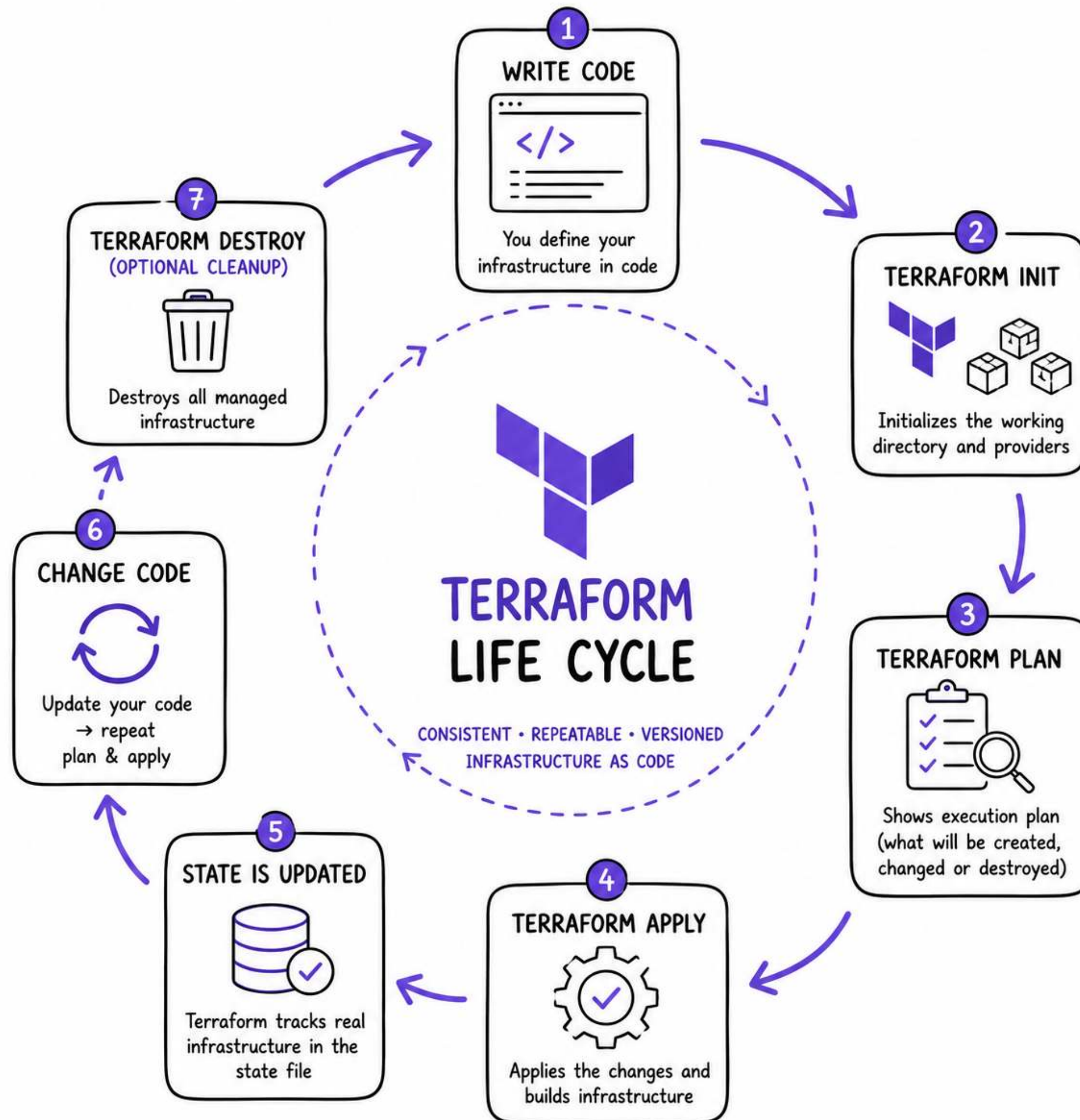


TERRAFORM BUILDS YOUR INFRASTRUCTURE

INFRASTRUCTURE CREATED!



DONE!



Terraform Init

This is what happens in the background when we us **terraform init**

1. **Downloads Providers** : Terraform checks your code. If you write the code for **AWS infra** then it downloads **AWS Plugin**
2. **Creates .terraform directory**: This folder contains:
 - a. downloaded providers
 - b. modules
 - c. cached files
3. **Reads configuration files**: Terraform scans your
 - a. **.tf files**
 - b. **modules**
 - c. **variables**
 - d. **providers**

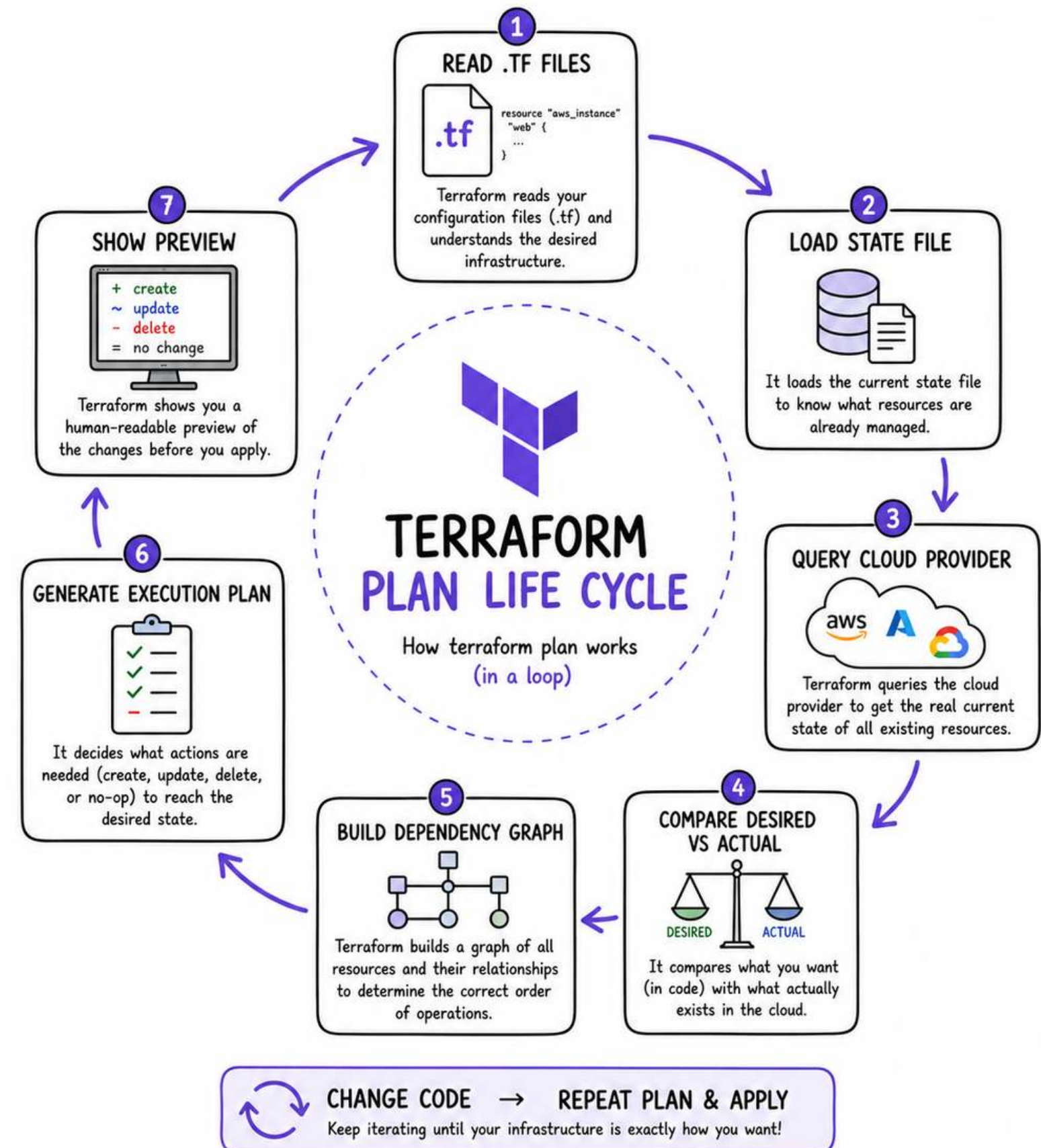
NOTE: It just scan your code, doesn't build any infra

Terraform Init

4. **Downloads modules (if used)** : It downloads those modules from registry or GitHub.
5. **Locks dependency versions** : Terraform ensures correct provider versions are used.
6. **Configures backend** : If your project uses remote state like S3 it will
 - a. connect to backend storage
 - b. configure state locking
 - c. prepare state file location

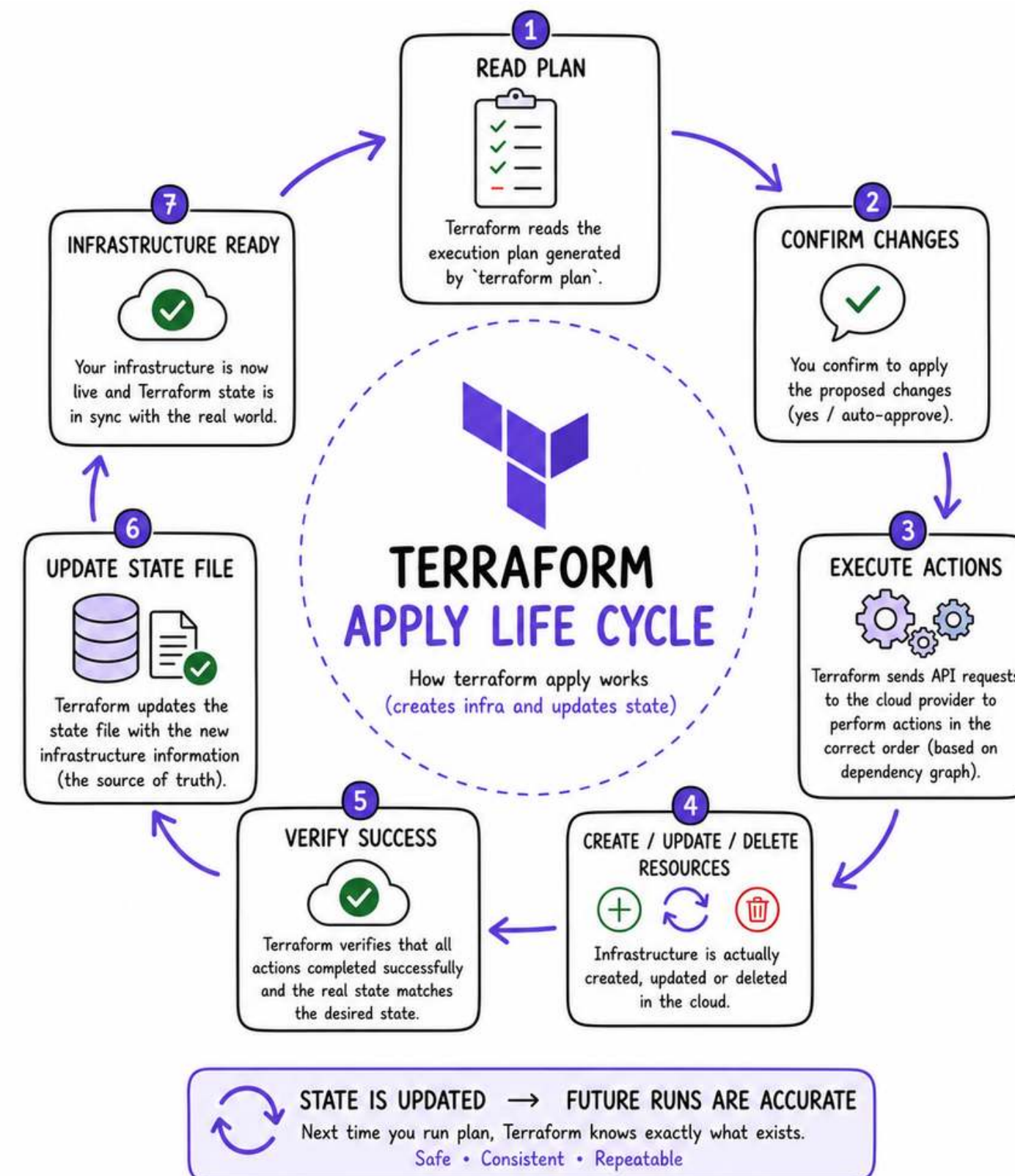
Terraform Plan

terraform plan compares what you want (your code) with what actually exists and calculates the changes needed.



Terraform Apply

terraform apply will apply those changes and updates it on statefile



Terraform Blocks

Terraform code is written using a language called HCL
Terraform code is built using blocks.

≡ TERRAFORM BLOCKS – AT A GLANCE ≡

8 Building Blocks of Every Terraform File

1 TERRAFORM BLOCK → SETUP



```
terraform {  
  required_version = ">= 1.0"  
}
```

Used to configure Terraform itself.

- Terraform version
- Provider versions
- Backend config

2 PROVIDER BLOCK → CONNECT



```
provider "aws" {  
  region = "ap-south-1"  
}
```

Tells Terraform which platform to talk to.

- AWS
- Azure
- GCP

3 RESOURCE BLOCK → CREATE



```
resource "aws_instance" "web" {  
  instance_type = "t2.micro"  
}
```

Defines infrastructure to create.

STRUCTURE:

```
resource "resource_type" "local_name"
```

★ Most Important!

4 VARIABLE BLOCK → INPUT



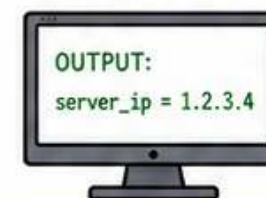
```
variable "server_size" {  
  default = "t2.micro"  
}
```

Makes code reusable and customizable.

USE:

```
instance_type = var.server_size
```

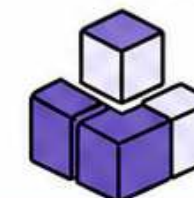
5 OUTPUT BLOCK → DISPLAY



```
output "server_ip" {  
  value = aws_instance.web.public_ip  
}
```

Shows values after apply. Useful for getting info like IPs, URLs, IDs, etc.

6 MODULE BLOCK → REUSE



```
module "network" {  
  source = "../modules/vpc"  
}
```

Reuses infrastructure code. Helps follow DRY principle and maintain consistency.

7 DATA BLOCK → READ

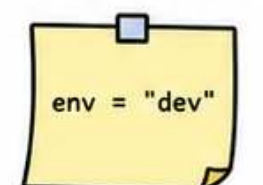


```
data "aws_ami" "ubuntu" {  
  most_recent = true  
}
```

Reads existing infrastructure or information.

i Does NOT create any resources.

8 LOCALS BLOCK → STORE (INTERNAL)



```
locals {  
  env = "dev"  
}
```

Stores temporary values for reuse within the module. Not changed from outside.

Terraform Code for EC2 Instance:

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
  ami           = "ami-0c55b159cbfaffe1f0"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "ExampleInstance"  
  }  
}
```

Code for EC2 Instance with Variables:

```
provider "aws" {  
  region = var.region  
}  
  
resource "aws_instance" "web" {  
  ami           = var.ami  
  instance_type = var.instance_type  
  tags = {  
    Name = "WebServer"  
  }  
}
```

main.tf

```
variable "ami" {  
  description = "The AMI to use for the instance"  
  type        = string  
  default     = "ami-0c94855ba95c71c99"  
}  
  
variable "instance_type" {  
  description = "The type of instance to use"  
  type        = string  
  default     = "t2.micro"  
}  
  
variable "region" {  
  description = "The AWS region to create the instance"  
  type        = string  
  default     = "us-east-1"  
}
```

variable.tf

Code for EC2 Instance using tfvars file:

Instead of passing the **default values** in variable.tf we can also use **terraform.tfvars** file. While using terraform plan or apply it will automatically loads the values from **terraform.tfvars** file

main.tf

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
    ami           = var.ami  
    instance_type = var.itype  
  
    tags = {  
        Name = var.iname  
    }  
}
```

var.tf

```
variable "ami" {  
    type = string  
}  
  
variable "itype" {  
    type = string  
}  
  
variable "iname" {  
    type = string  
}
```

terraform.tfvars

```
ami      = "ami-0741dc526e1106ae5"  
itype    = "t3.micro"  
iname    = "mustafa"
```

Code for EC2 Instance using tfvars in json file:

We can also pass those values in json format

main.tf

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
    ami           = var.ami  
    instance_type = var.itype  
  
    tags = {  
        Name = var.iname  
    }  
}
```

var.tf

```
variable "ami" {  
    type = string  
}  
  
variable "itype" {  
    type = string  
}  
  
variable "iname" {  
    type = string  
}
```

terraform.tfvars

```
{  
    "ami": "ami-fd89jo37shs",  
    "itype": "t2.micro",  
    "iname": "web-server"  
}
```

Code for EC2 Instance using Auto tfvars:

When you have multiple tf var files like **dev.auto.tfvars**, **test.auto.tfvars**, **prod.auto.tfvars** we can use **terraform apply -var-file=filename**. so that terraform will automatically takes the values from those files and create the resources.

main.tf

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
  ami           = var.ami  
  instance_type = var.itype  
  
  tags = {  
    Name = var.iname  
  }  
}
```

var.tf

```
variable "ami" {  
  type = string  
}  
  
variable "itype" {  
  type = string  
}  
  
variable "iname" {  
  type = string  
}
```

```
ami    = "ami-0741dc526e1106ae5"  
itype  = "c7i-flex.large"  
iname  = "devserver"
```

dev.auto.tfvars

```
ami    = "ami-0741dc526e1106ae5"  
itype  = "m7i-flex.large"  
iname  = "prodserver"
```

prod.auto.tfvars

Code for EC2 Instance using CLI Variables:

Instead of passing the values on files, we can directly pass those values dynamically
ex: **terraform apply -var="instance_type=t2.micro"**

main.tf

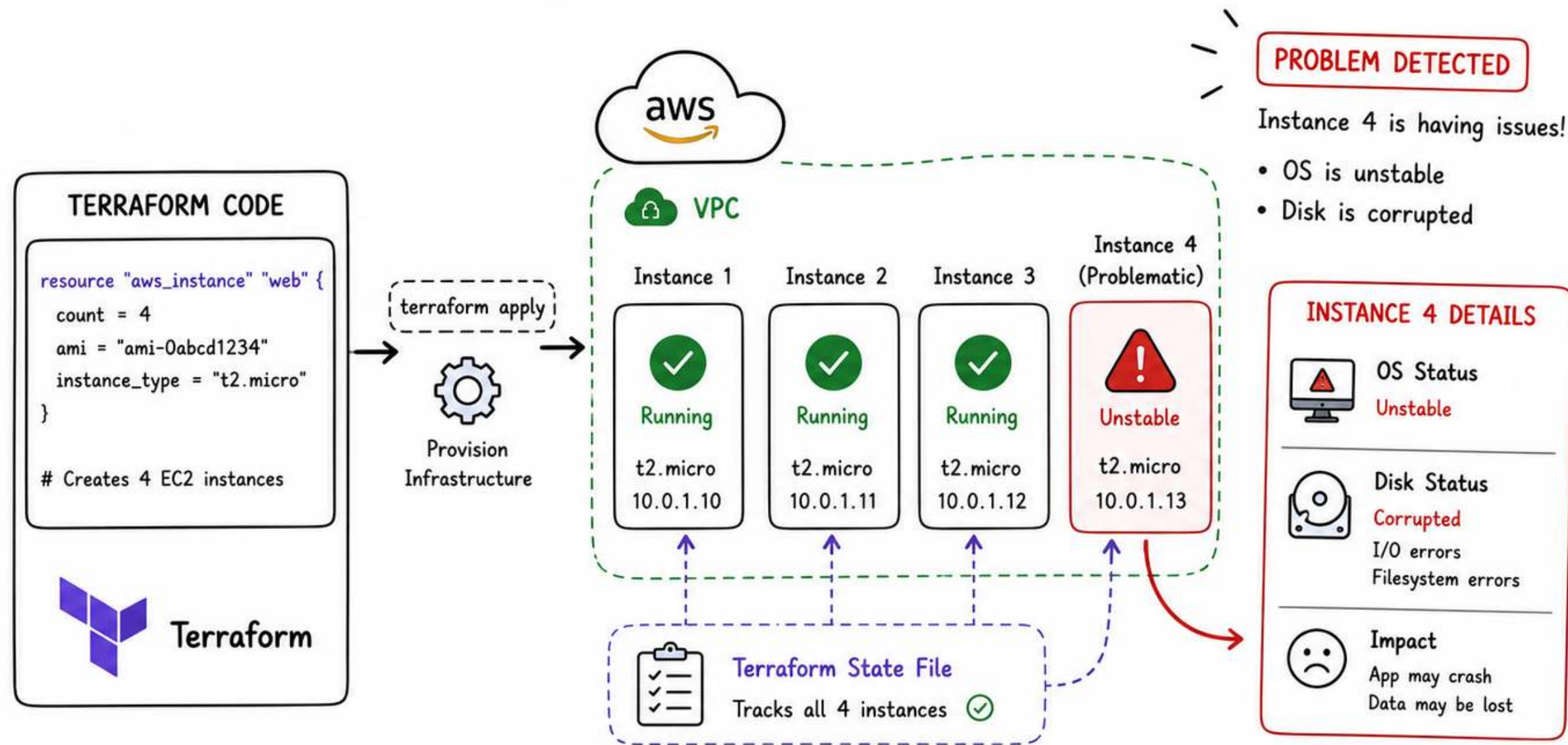
```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
  ami           = var.ami  
  instance_type = var.itype  
  
  tags = {  
    Name = var.iname  
  }  
}
```

var.tf

```
variable "ami" {  
  type = string  
}  
  
variable "itype" {  
  type = string  
}  
  
variable "iname" {  
  type = string  
}
```

terraform apply -var="itype=t2.micro" -var="ami=ami-83jhdy03un" -var="iname=web-server"

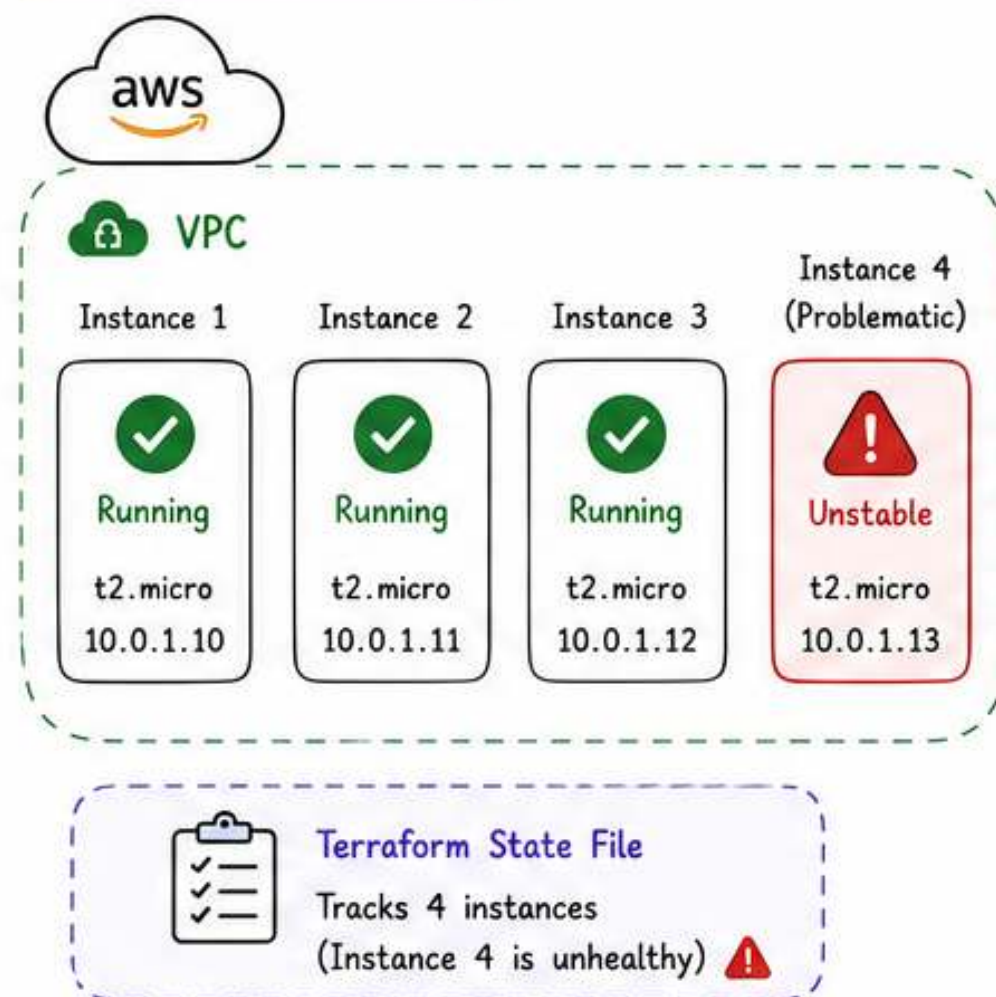
TERRAFORM REPLACE:



SUMMARY

Terraform created 4 instances.
Instance 4 is unhealthy due to OS instability and disk corruption.

CURRENT STATE



Instance 4 is unhealthy.
OS is unstable and disk is corrupted.

Let's use
terraform replace
to recreate it!



DEVOPS ENGINEER

TERRAFORM CODE (SAME CONFIG)

```
resource "aws_instance" "web" {  
  count      = 4  
  ami        = "ami-0abcd1234"  
  instance_type = "t2.micro"  
  ...  
}
```

No changes in configuration

PLAN: REPLACE INSTANCE 4

```
$ terraform plan -replace=aws_instance.web[3]
```

Terraform will perform the following actions:

-/+ replace aws_instance.web[3]
(1 change)

Plan: 1 to add, 1 to destroy, 0 to change.

APPLY THE PLAN

```
$ terraform apply -replace=aws_instance.web[3]
```

Apply complete! Resources: 1 added, 1 destroyed.

WHAT HAPPENS WHEN TERRAFORM REPLACE IS EXECUTED?

1 DESTROY OLD INSTANCE 4



2 CREATE NEW INSTANCE WITH SAME CONFIG



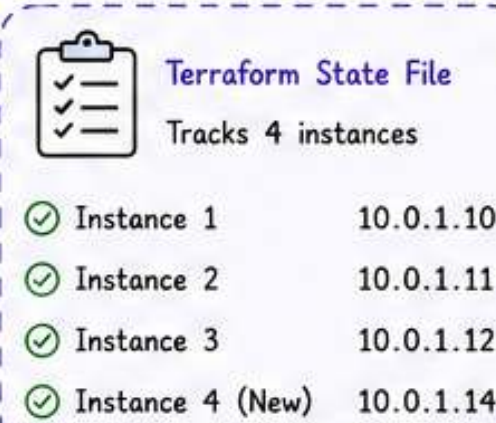
Creating new instance...

3 NEW INSTANCE IS HEALTHY

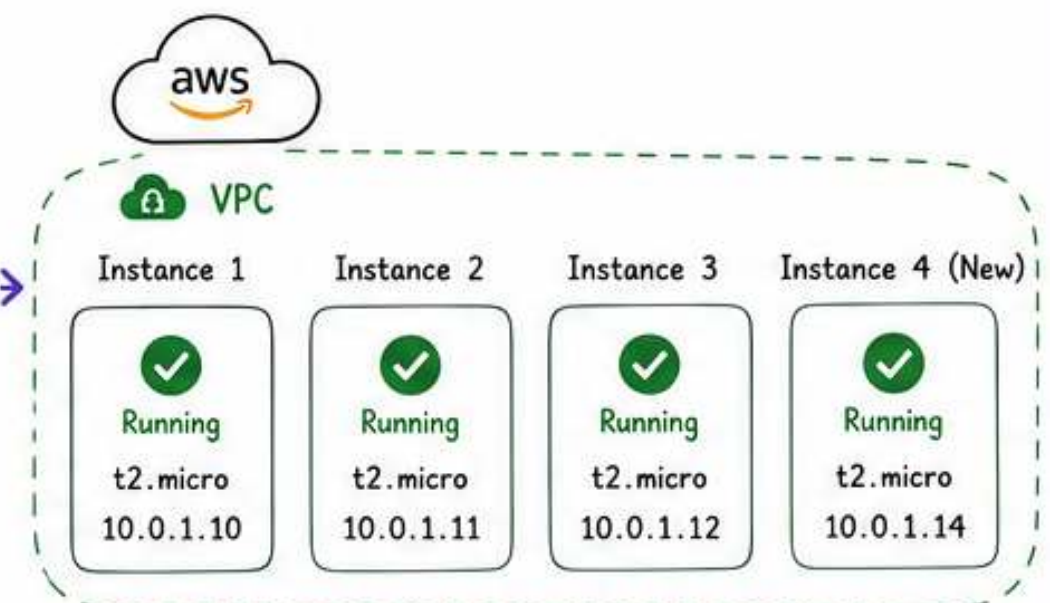


New instance is up and running!

4 STATE FILE IS UPDATED

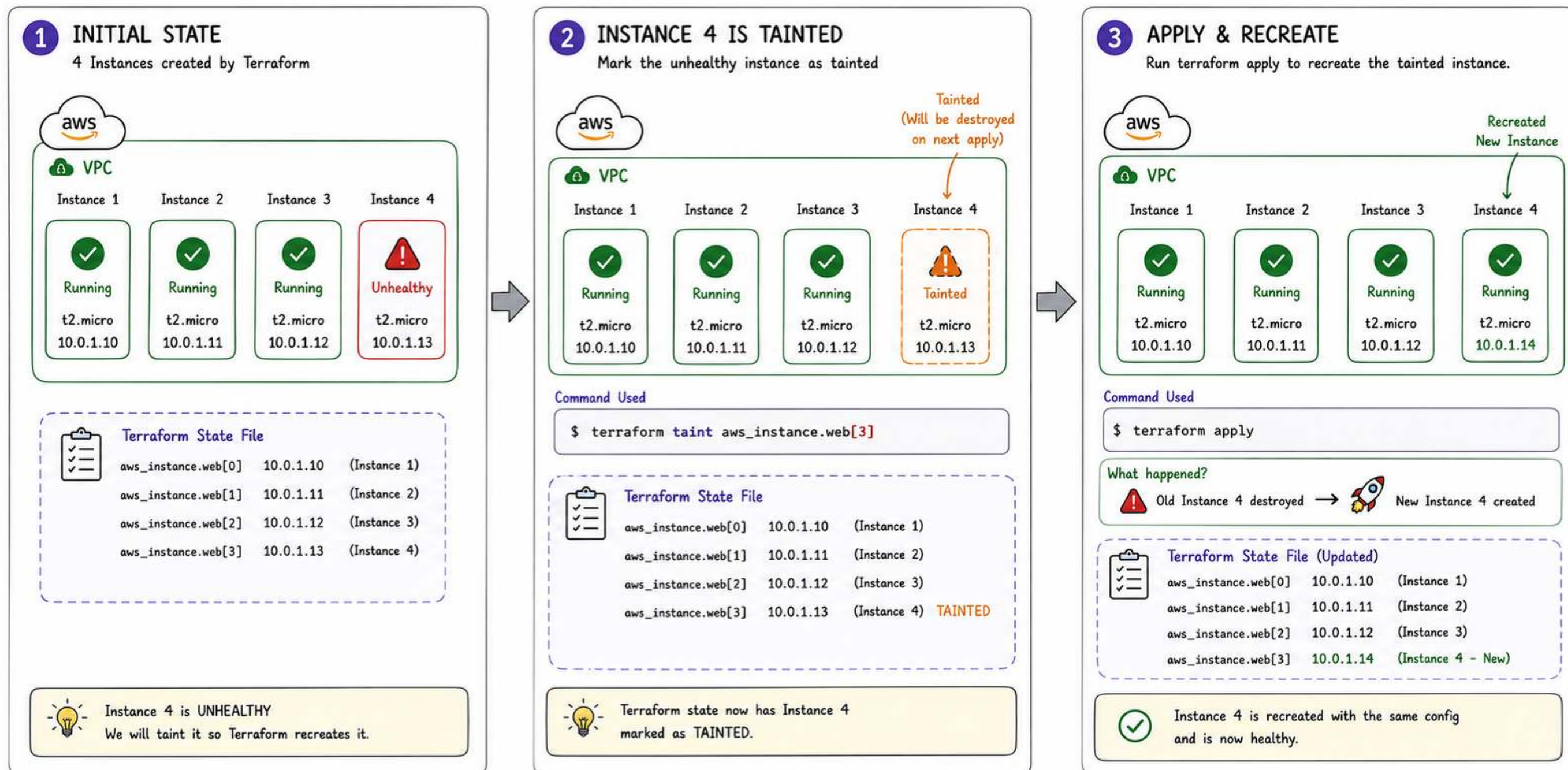


5 NEW DESIRED STATE



TERRAFORM TAINT:

Taint is also used to recreate the resources but taint is the old command & replace is the new command in terraform



NEW DEVOPS ENGINEER JOINS THE STARTUP!



A NEW BEGINNING

A new DevOps Engineer has joined the startup team today!



MISSION

Automate everything
Deploy faster
Ensure reliability
Scale with confidence

EXCITED TO BUILD
SCALABLE & RELIABLE
INFRASTRUCTURE!

DEVOPS

WHAT HE WILL WORK ON



INFRASTRUCTURE
AS CODE



AUTOMATION &
CI/CD



MONITORING &
RELIABILITY



SCALABILITY &
PERFORMANCE

WELCOME TO THE TEAM! 🚀

TOOLS & TECHNOLOGIES



docker



kubernetes



Terraform



aws



GitHub



Jenkins



Prometheus



Grafana



GOAL

Help the startup move fast,
ship often, and build a
strong DevOps culture!



TODAY IS DAY 1

- ✓ Set up workstation
- ✓ Access to environments
- ✓ Understand the stack
- ✓ Start building awesome things!



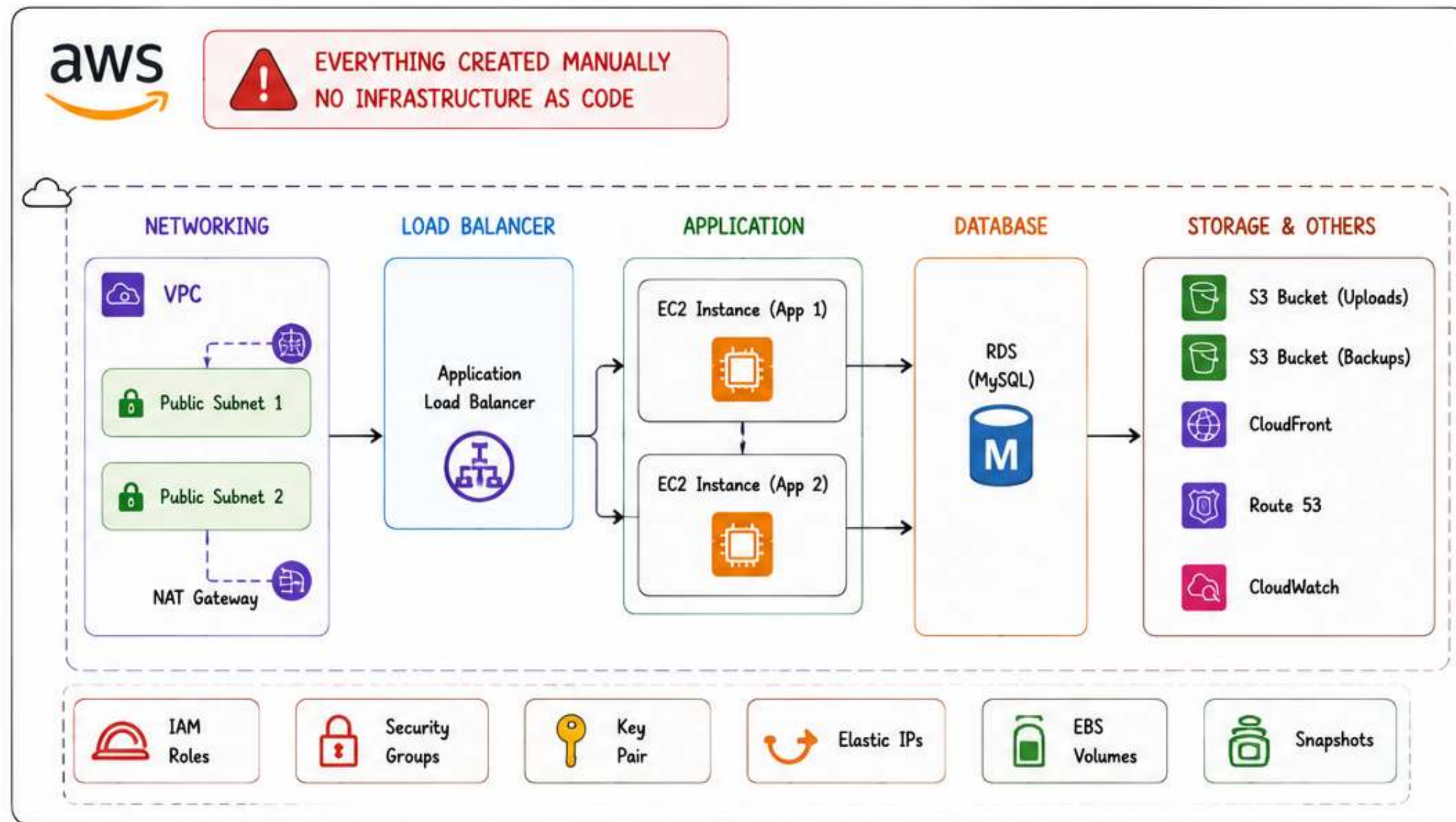
DAY 2: A SHOCKING DISCOVERY!

The new DevOps Engineer was shocked to see the entire application architecture is built manually on AWS!



NEW DEVOPS ENGINEER

- ✗ Expected Infrastructure as Code
- ✗ Expected Automation
- ✗ Expected Reusability
- ✗ Expected Best Practices



NEW DEVOPS ENGINEER

Hey team,
why is the entire application
architecture built **manually**
on AWS?

OUR APPLICATION ARCHITECTURE



WHY MANUAL?

- ✗ No Infrastructure as Code
- ⚙ Hard to track changes
- 👥 Difficult to reproduce
- 🕒 Takes more time
- ⚠ High risk of human error

TEAM RESPONSE

We usually follow this approach from **many years**.

We started when the team
was small. It was faster
to create resources
manually.

Everyone here is
comfortable with this
process.

It works for us.
We've been doing it
this way for years
without major issues.

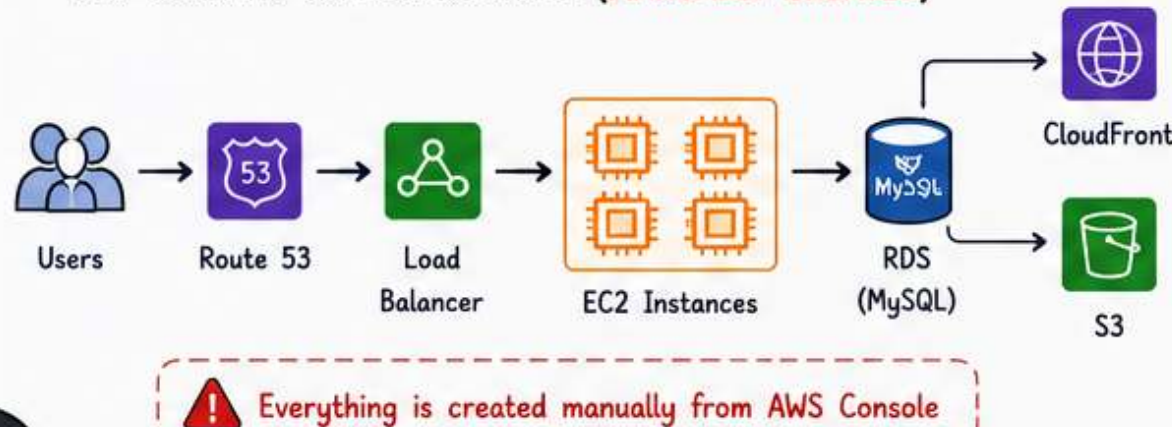
OUR REASONING

- 📅 We've been following this approach for many years.
- 👥 The team is familiar and comfortable with it.
- 🕒 It was quick and easy in the beginning.
- 🛡 It has worked for us so far.
- 💡 Change feels risky. Sticking to what we know feels safer.

NEW DEVOPS ENGINEER

I think we should use **Terraform** to automate our infrastructure. It will help us be faster, consistent and reduce manual errors!

OUR CURRENT INFRASTRUCTURE (MANUALLY CREATED)



WHY TERRAFORM?

- Infrastructure as Code
- Consistency & Repeatability
- Faster Provisioning
- Team Collaboration
- Better Security & Compliance
- Scalability & Maintainability

TEAM RESPONSE

Let's use **Terraform** and automate our work!

We have **already** created the entire infrastructure manually.

If we implement Terraform now, it will take **a lot of time**.

We can't take the risk now. We have **tight deadlines** to deliver the project.



We understand the concerns. Let's plan a safe and gradual approach to bring Terraform without affecting our deadlines. We can start small and move step by step.

OUR CONCERNS

- ! Learning Terraform will take time
- ! Migrating existing resources will take time
- ! High chance of breaking things
- ! Project deadline is very tight
- ! We need to deliver features, not spend time on tooling right now



OUR PRIORITY RIGHT NOW

Deliver the project on time and keep the application stable.

TEAM DISCUSSION

You don't worry, I will **import** all the existing infra to **Terraform**.

OUR INFRASTRUCTURE (MANUALLY CREATED)

The diagram shows a sequence of AWS services: Route 53 (purple shield icon) points to Load Balancer (purple server icon), which points to EC2 Instances (orange server rack icon), which points to RDS (MySQL) (blue database icon), which points to S3 (green bucket icon). From S3, arrows point to CloudFront (purple globe icon) and Others (CloudWatch, SNS, etc.) (purple cloud icon).

TEAM'S CONCERN

- ⚠️ We already created the entire infra manually.
- ⚠️ If we implement Terraform now, it will take a lot of time.
- ⚠️ We can't take the risk now.
- ⚠️ Our project deadline is very tight.

TEAM REACTION

How will you import everything?

Is that even possible?

Will it break anything?

How will Terraform know about all the existing resources?

Will it take more time?

DEVOPS ENGINEER'S REPLY

The DevOps engineer, wearing a purple hoodie, is shown on the left with a confident expression and arms crossed. The other five team members are shown on the right, each with a question mark above their head, indicating they are still confused or skeptical.

TERRAFORM IMPORT:

Bring existing infrastructure under Terraform management without recreating it.

Example of Import Existing EC2

Suppose EC2 already exists:

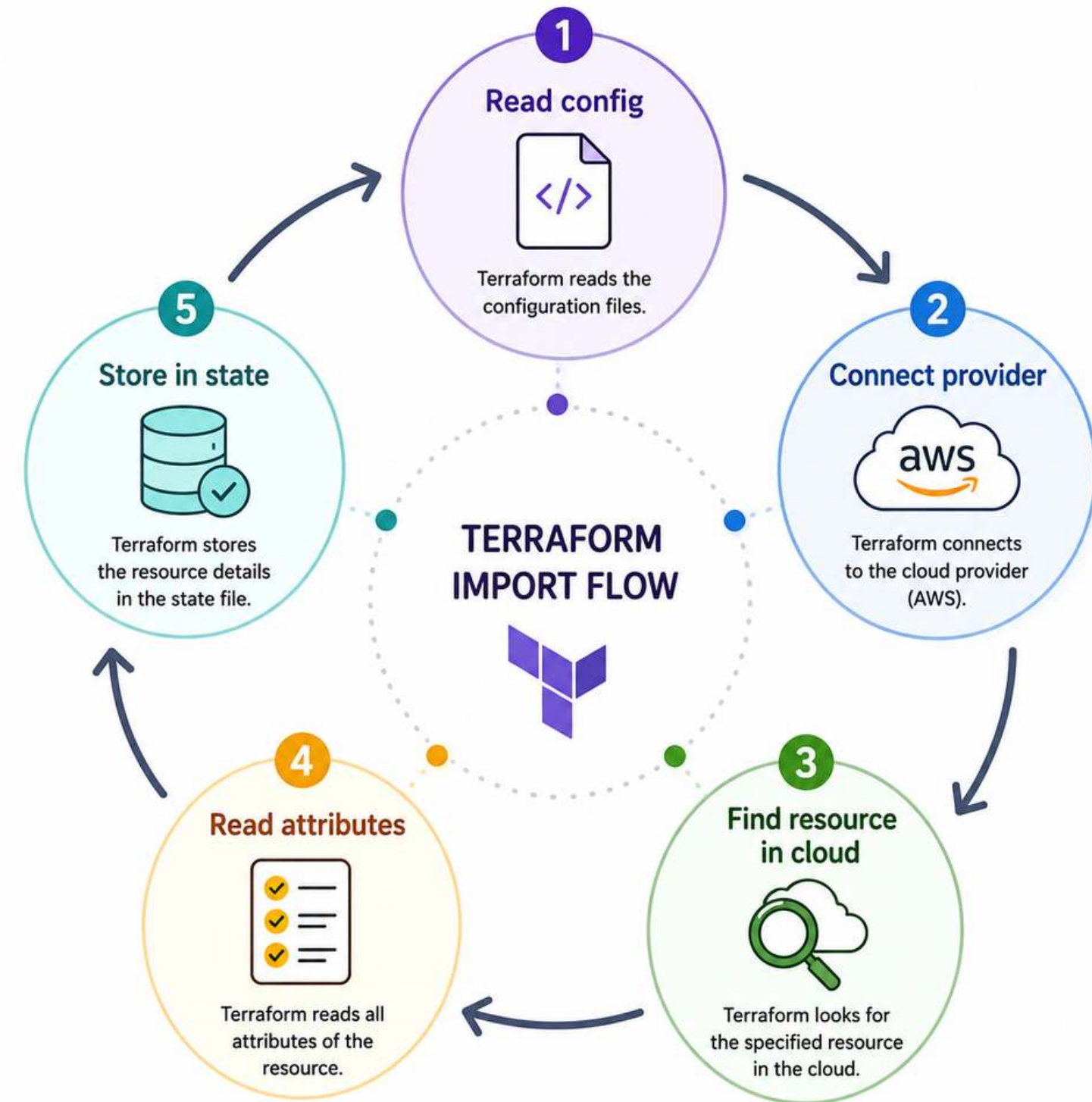
Instance ID:
i-123456

Write Terraform config first:

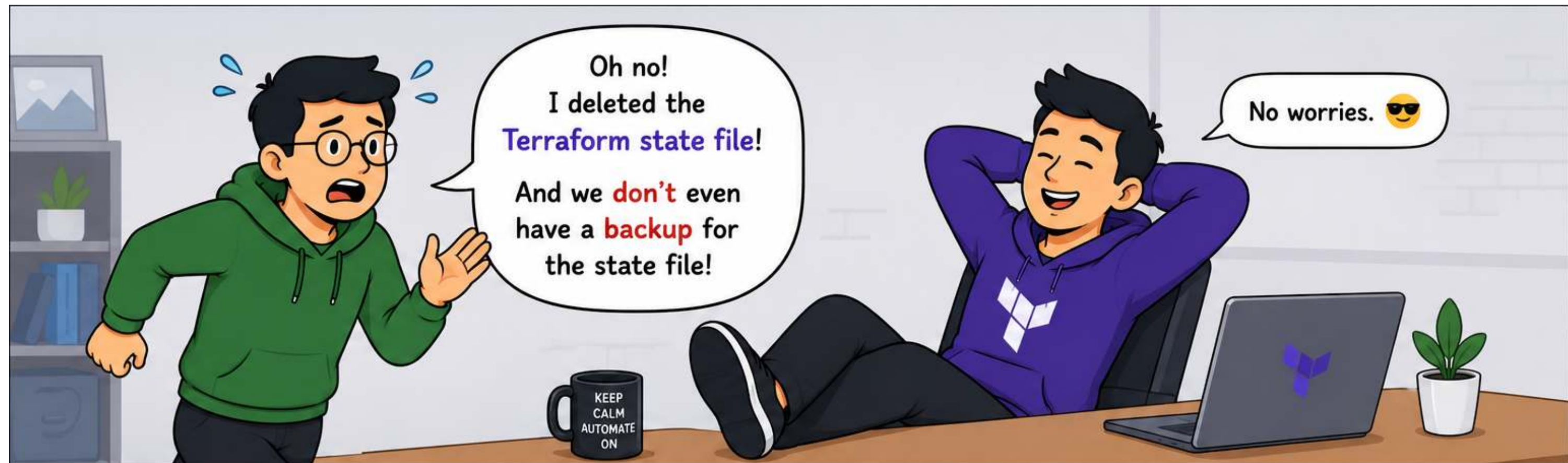
```
resource "aws_instance" "web" {  
}
```

Now import:

```
terraform import aws_instance.web i-123456
```







4 RUNNING TERRAFORM IMPORT

He came to work and simply used terraform import again.



5 RESOURCES IMPORTED

Terraform read all the existing resources from the cloud.



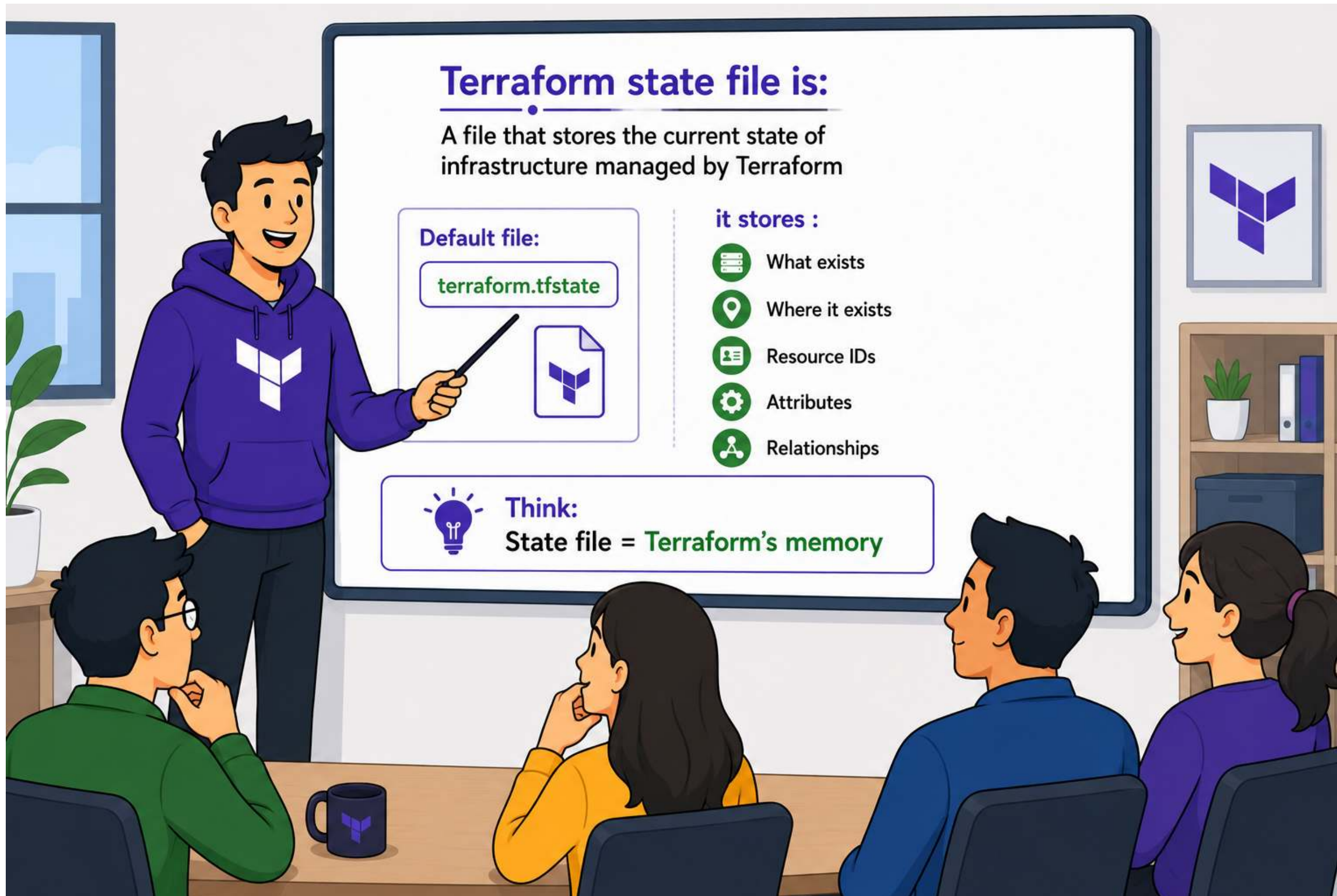
6 STATE FILE RESTORED

State file is back and everyone is happy!



✓ Infrastructure is back under control with Terraform!

TERRAFORM STATEFILE:



Why state file is important

Without state:

Terraform cannot know:

- What exists?
- What changed?
- What to update?

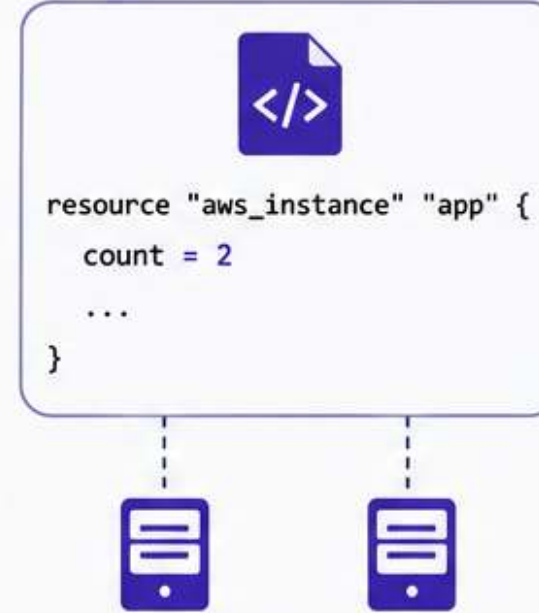
Terraform would keep creating duplicates.

Example:

Current:
1 server exists



Code:
Want 2 servers



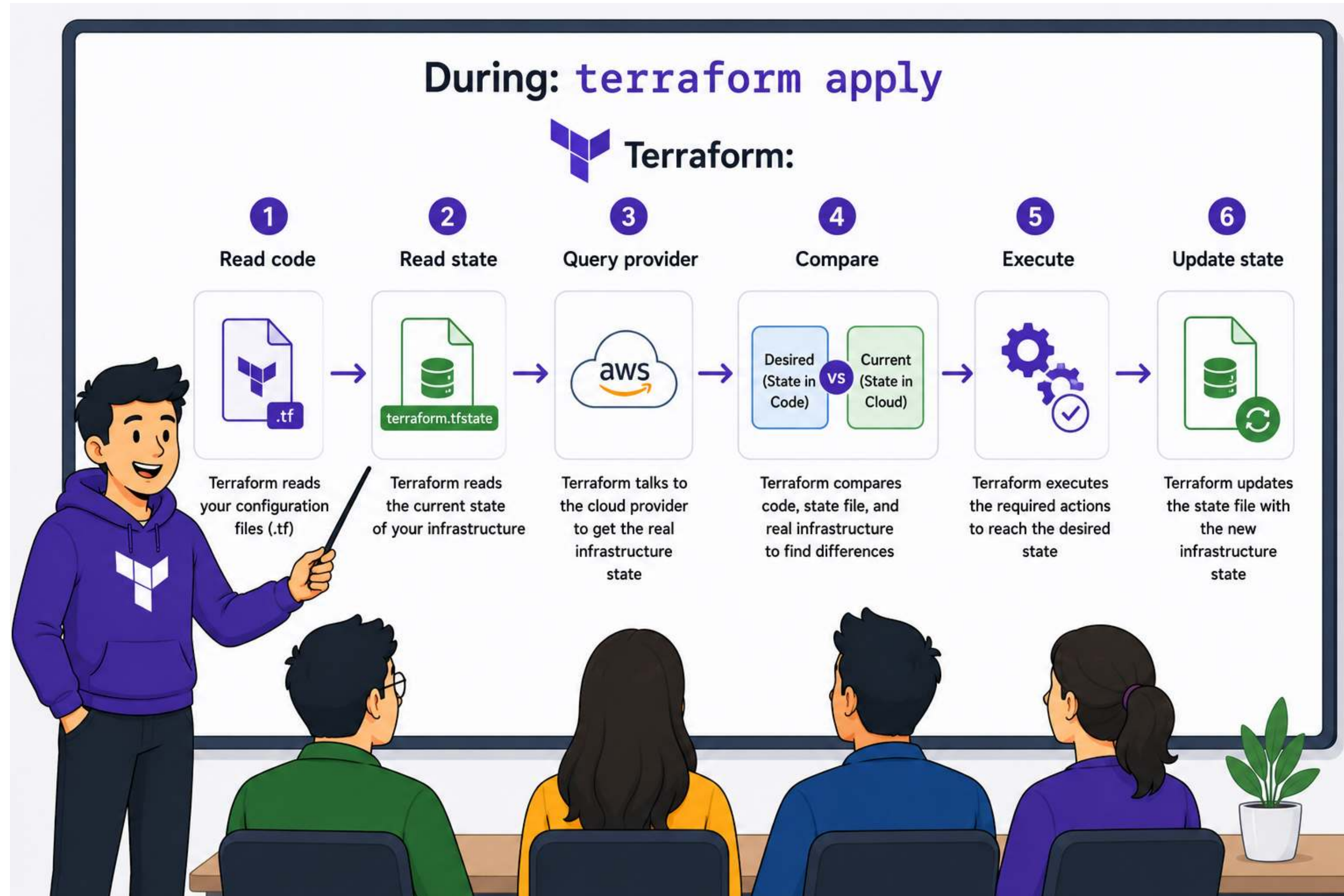
State helps Terraform calculate:
Add only 1 server



State file shows what exists.

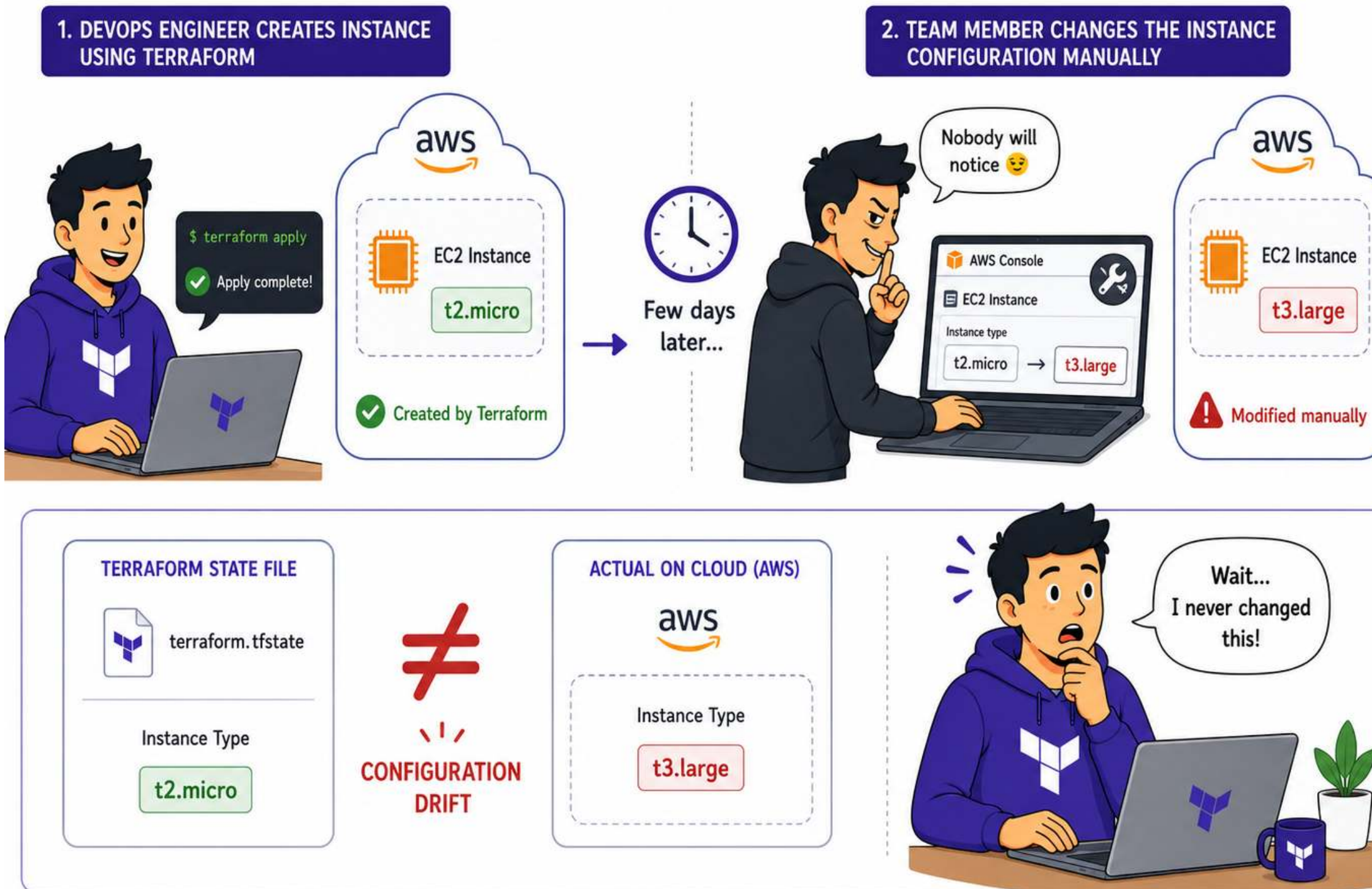
Terraform compares it with code and makes the **right change**.

What happens internally?



Statefile Helps you in drift detection

Terraform drift means:



How Terraform detects drift ★★ ★



MAIN METHOD:

>_ terraform plan

This command helps Terraform detect drift.

TERRAFORM INTERNALLY:

1. Read code



Terraform reads your configuration files (code).

2. Read state



Terraform reads the current state from the state file.

3. Query cloud



Terraform queries the cloud provider to get the real infrastructure state.

4. Compare



Terraform compares the code (desired state) with the real state.

5. Show differences



Terraform shows the differences (drift) if any.

EXAMPLE OUTPUT:

```
$ terraform plan
Terraform will perform the following actions:

~ update in-place
  aws_instance.example

  ~ instance_type:
    - "t3.large"
    + "t2.micro"

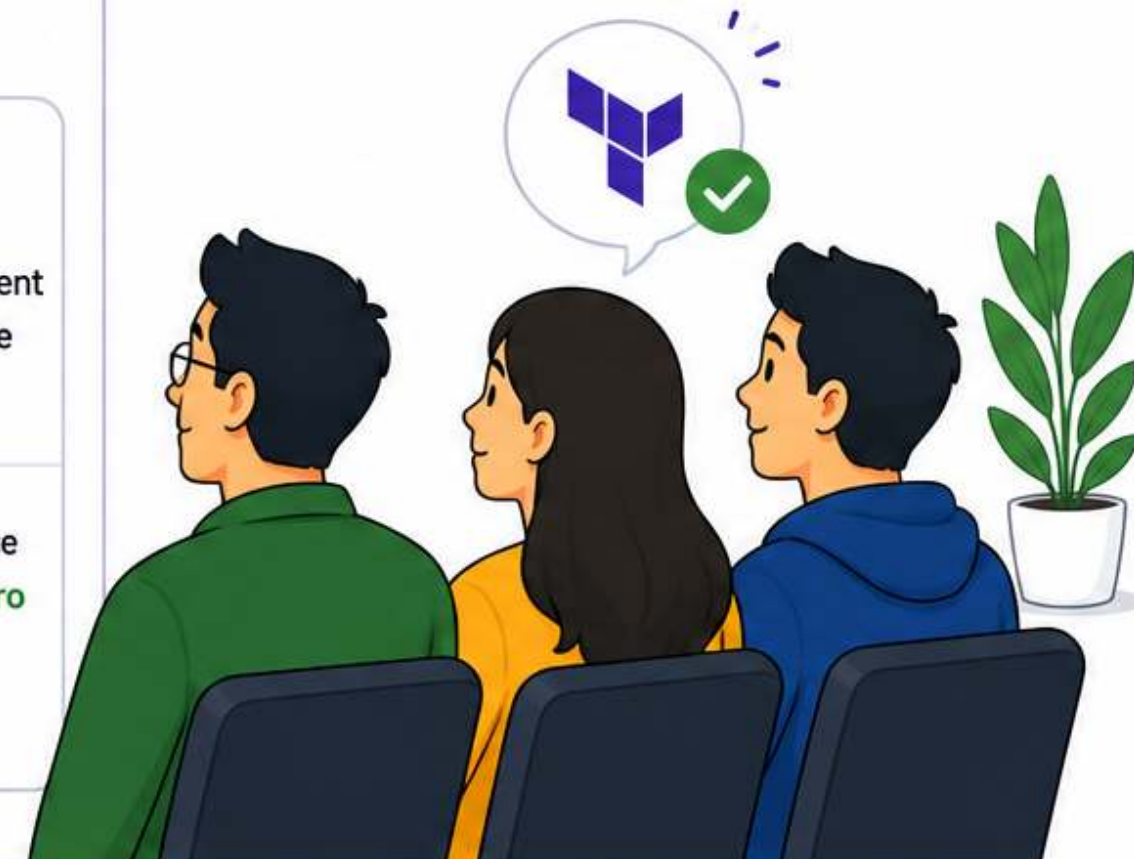
Plan: 0 to add, 1 to change, 0 to destroy.
```

MEANING:



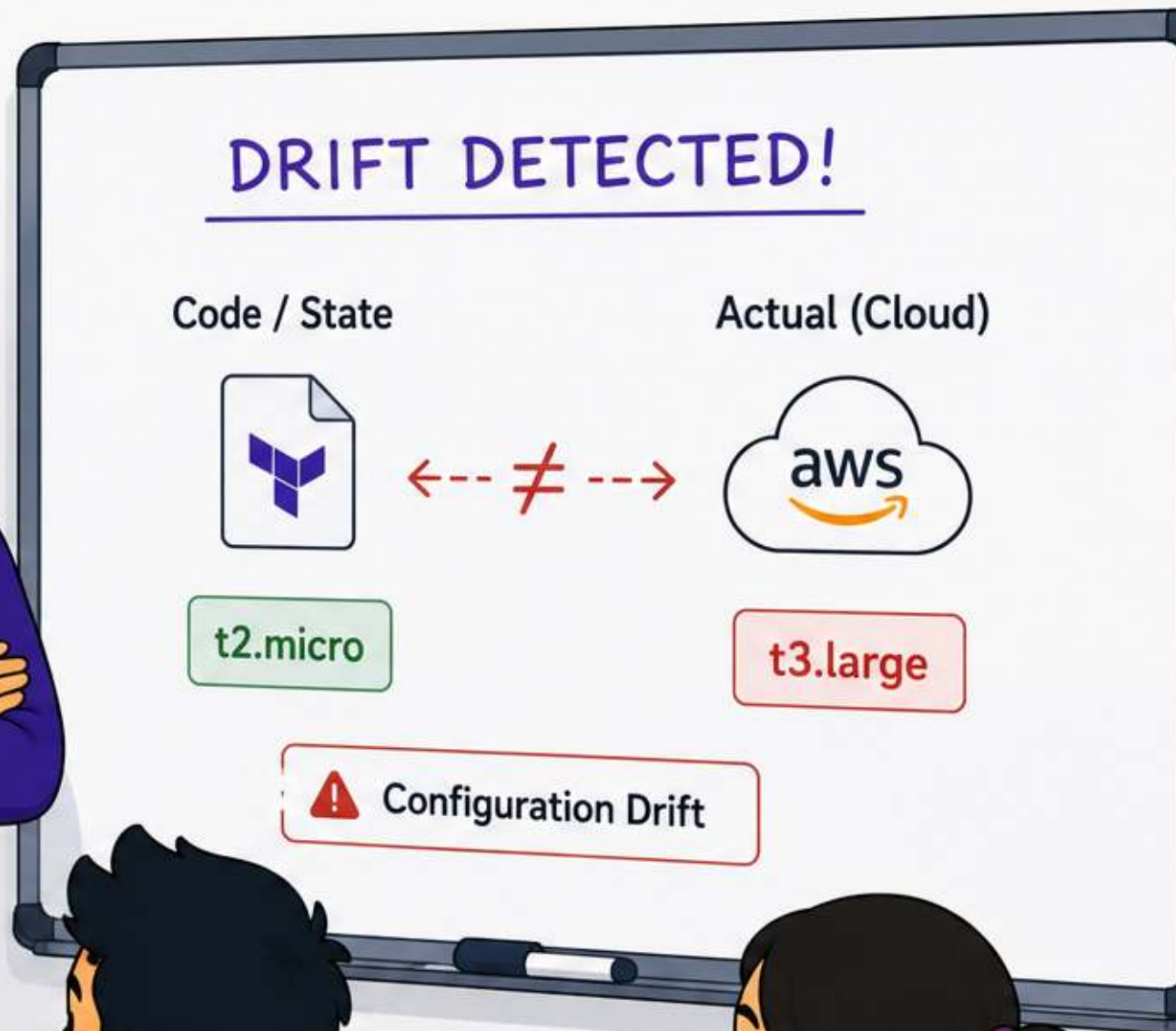
Terraform detected that the real infrastructure (**t3.large**) is different from what is defined in your code (**t2.micro**).

Terraform will update the instance in-place from **t3.large** to **t2.micro** to match your code.





What to do
after detecting
drift?





It depends
on **scenarios**.

It depends on **scenarios**

SCENARIO-1 **Accept Cloud Changes** ✓

When manual changes in cloud are correct and should be kept.

1. Update Terraform code

```
resource "aws_instance" "app" {  
  ami      = "ami-123456"  
  instance_type = "t2.micro"  
}
```



```
resource "aws_instance" "app" {  
  ami      = "ami-123456"  
  instance_type = "t3.large" # updated  
}
```

2. Example

Current cloud:



Update:

```
instance_type = "t3.large"
```

3. Apply

Run **terraform apply**
to update the state
and keep everything
in sync.



**Infrastructure and
state are now aligned.**



In short: You accept the real (cloud) changes and update code to match.



It depends
on **scenarios**.

It depends on **scenarios**

SCENARIO-2

Revert Cloud



When manual changes in cloud are NOT correct and should be reverted.

1. Keep code unchanged

Your Terraform code defines the desired state.

```
resource "aws_instance" "app" {  
  ami      = "ami-123456"  
  instance_type = "t2.micro"  
}
```



Desired in code:
instance_type = "t2.micro"

2. Run terraform apply

Terraform detects the drift and reverts it.

```
$ terraform apply
```



Terraform detects drift



Updates the infrastructure



Restores the desired state

3. Terraform restores desired state

Actual cloud matches the code.

Before (drifted)



After (reverted)



In short: You keep the **code** as it is, and Terraform brings back to match.



It depends
on scenarios.

It depends on scenarios

SCENARIO-3

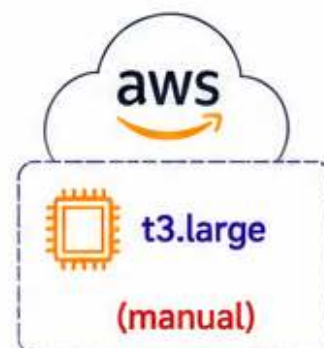
Import Missing Resource



If someone created a resource manually in the cloud, and it's not in Terraform state.

1 Resource created manually

Someone created an EC2 instance manually.



! Not in Terraform state
(Unknown to Terraform)

2 Import the resource into Terraform

Use terraform import to bring the existing resource into Terraform state.

```
$ terraform import  
aws_instance.web  
i-0abcd1234efgh5678
```

✓ Terraform now knows
about this resource.

3 Verify with terraform plan

Run plan to verify the resource.

```
$ terraform plan  
  
No changes.  
Your infrastructure  
matches the  
configuration.
```

✓ No drift after import
(if configs match).

4 Manage from now on

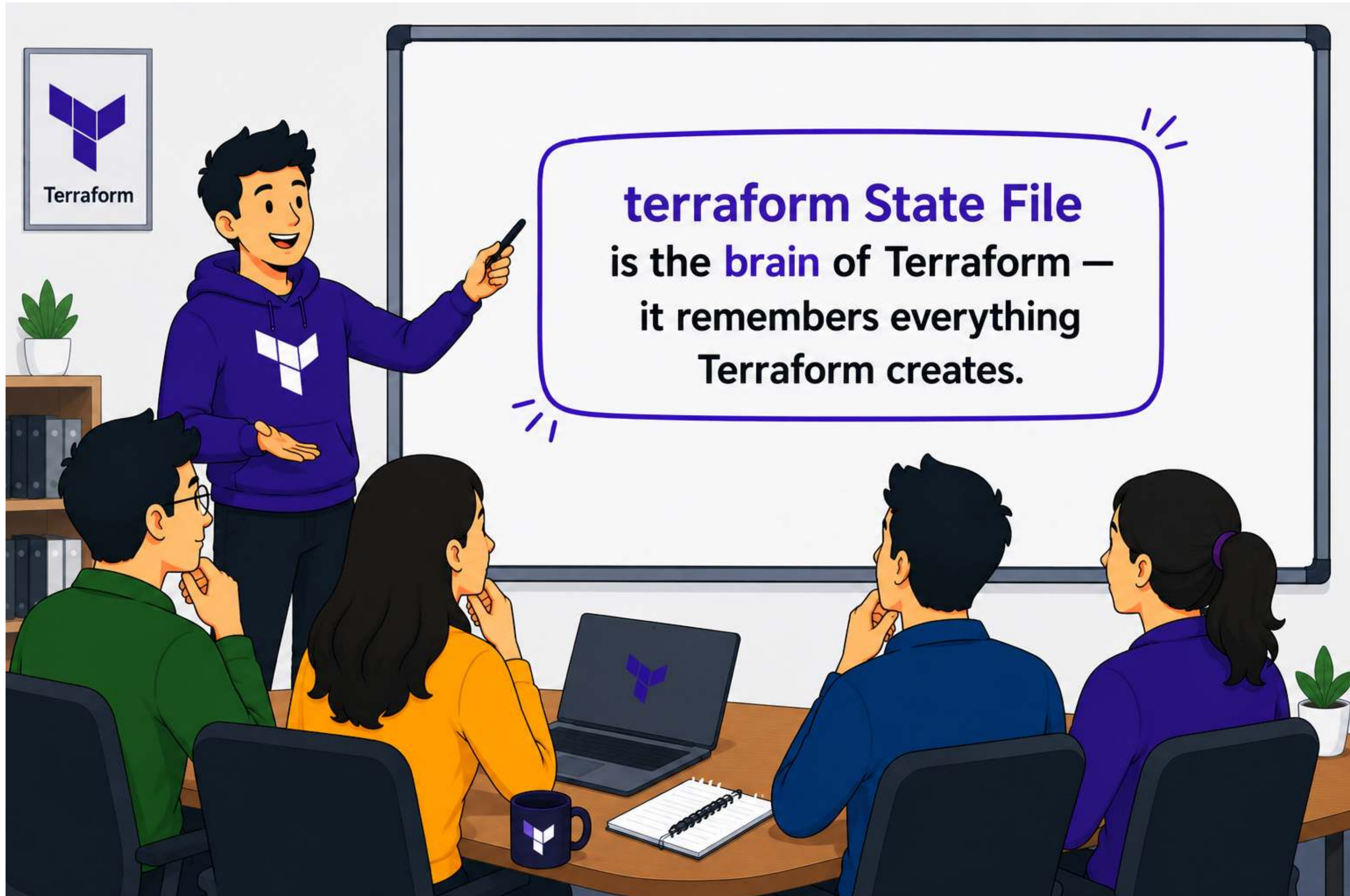
Now Terraform will manage this resource.

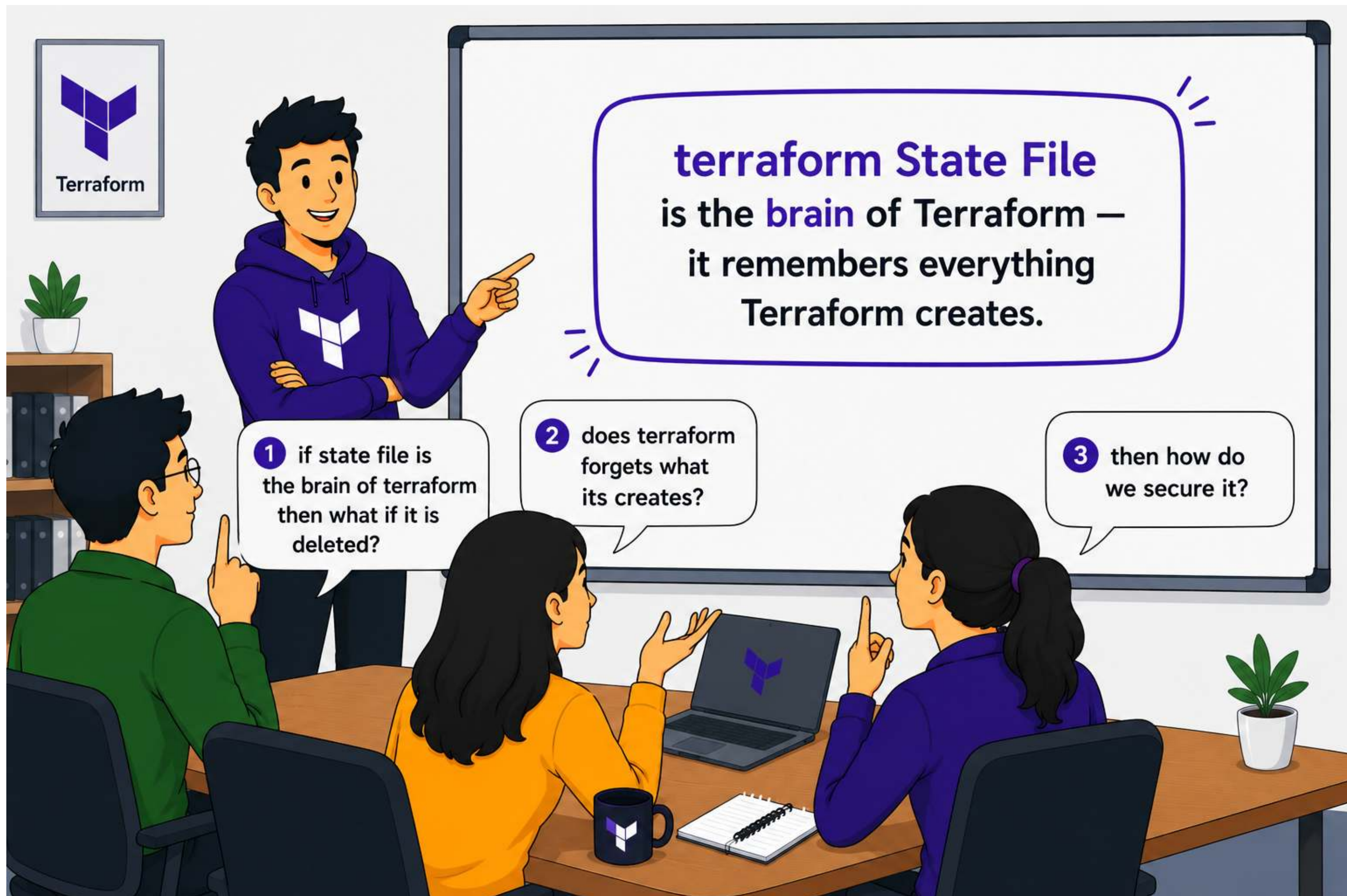


✓ Fully managed
by Terraform.



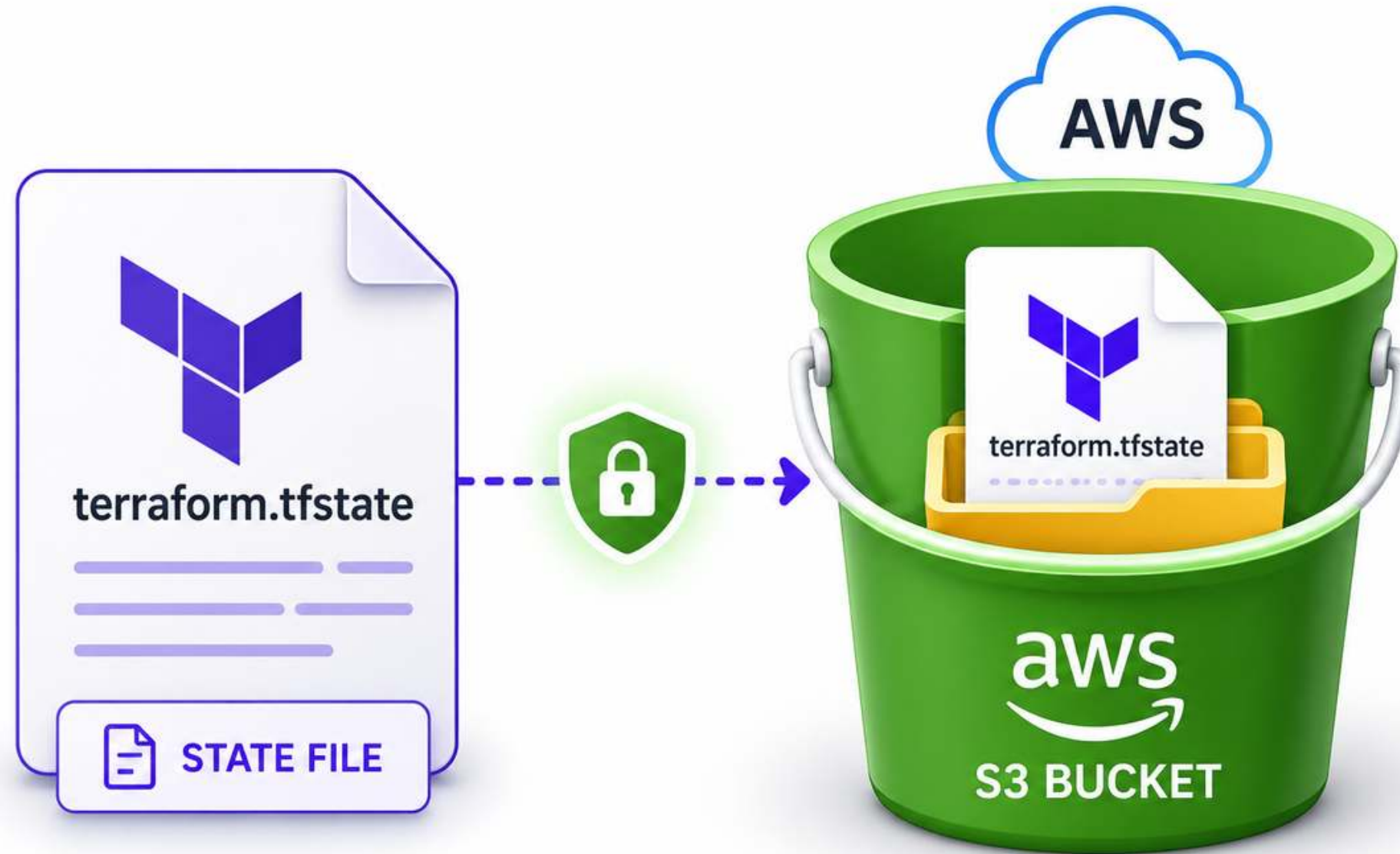
Import brings existing resources under Terraform management without recreating them.





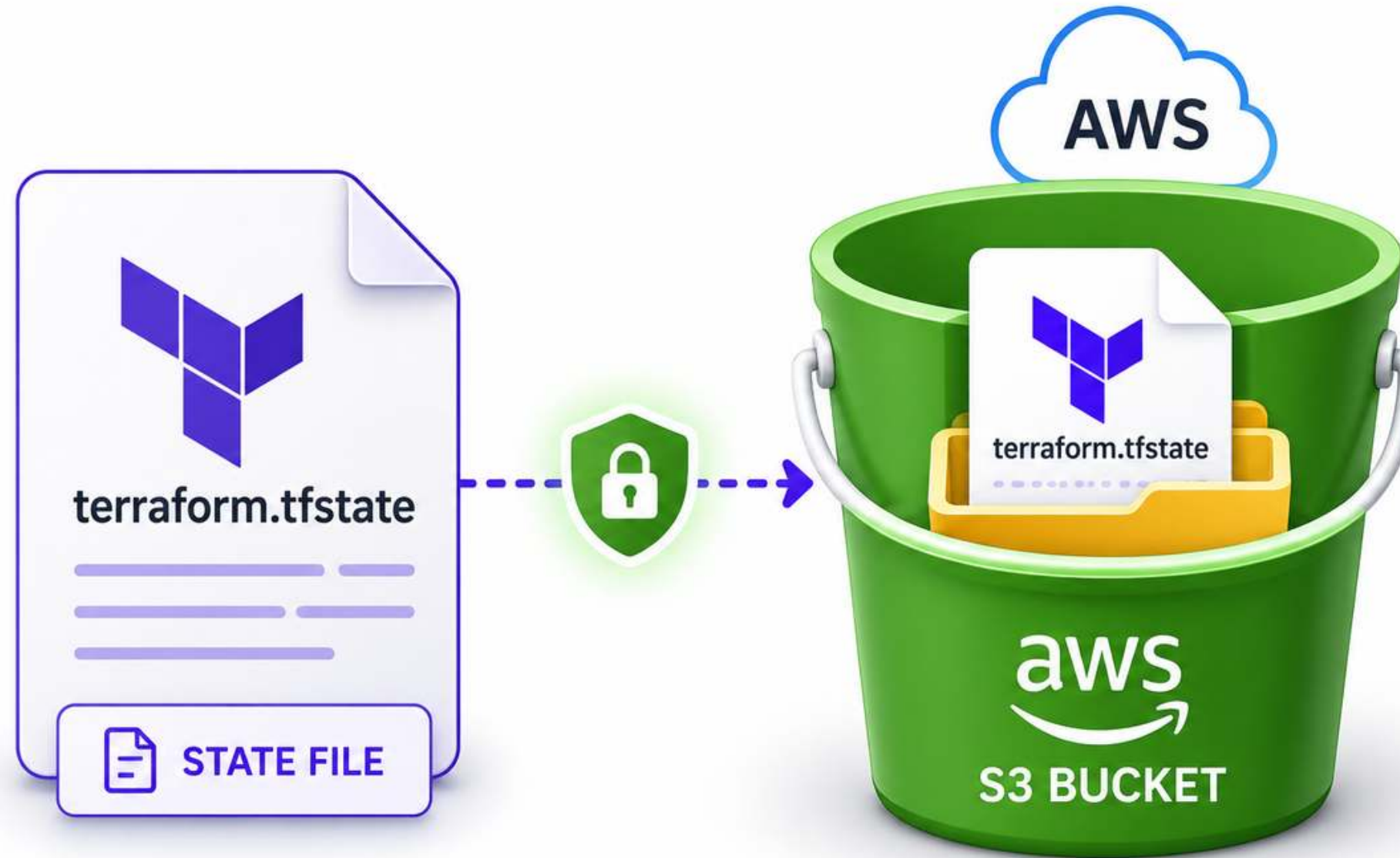
How to secure statefile:

1. Store State Remotely (Most Important)

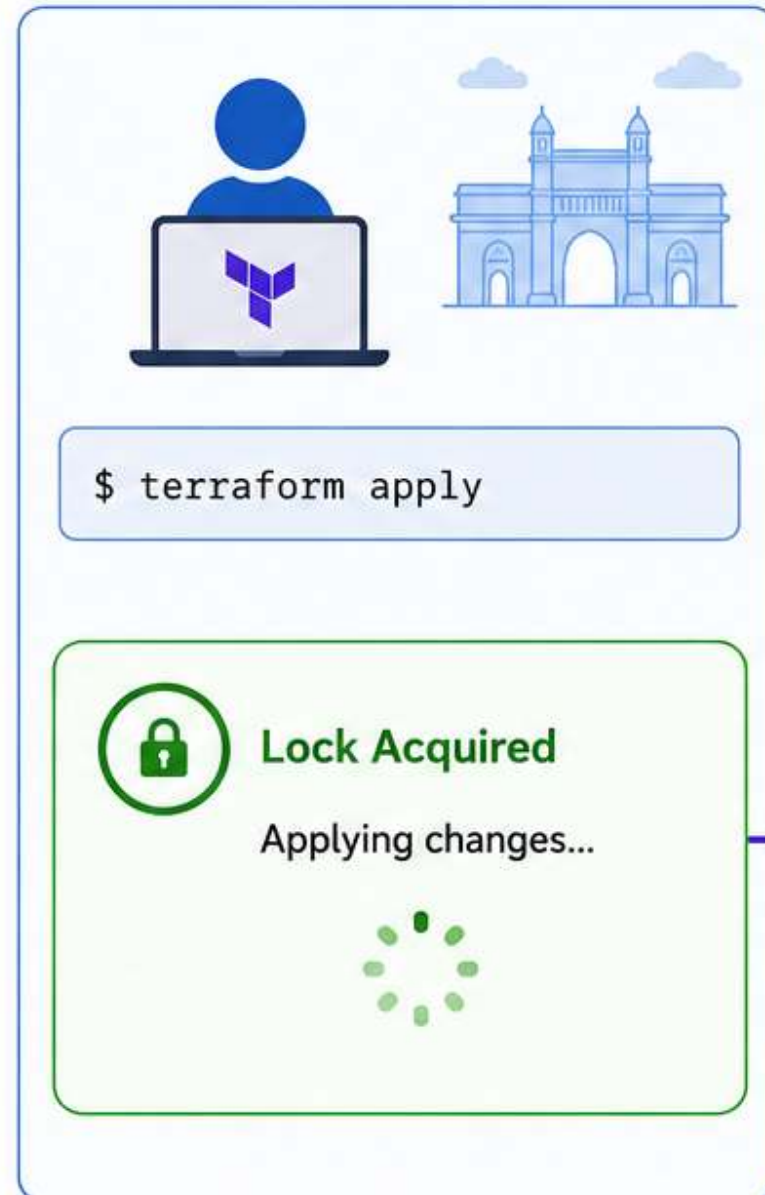


How to secure statefile:

1. Store State Remotely (Most Important)



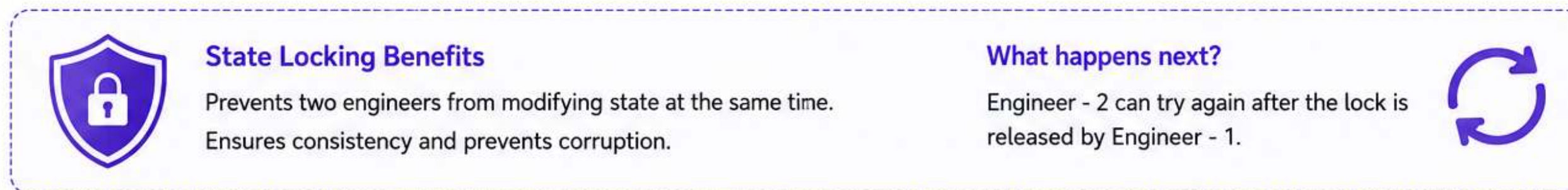
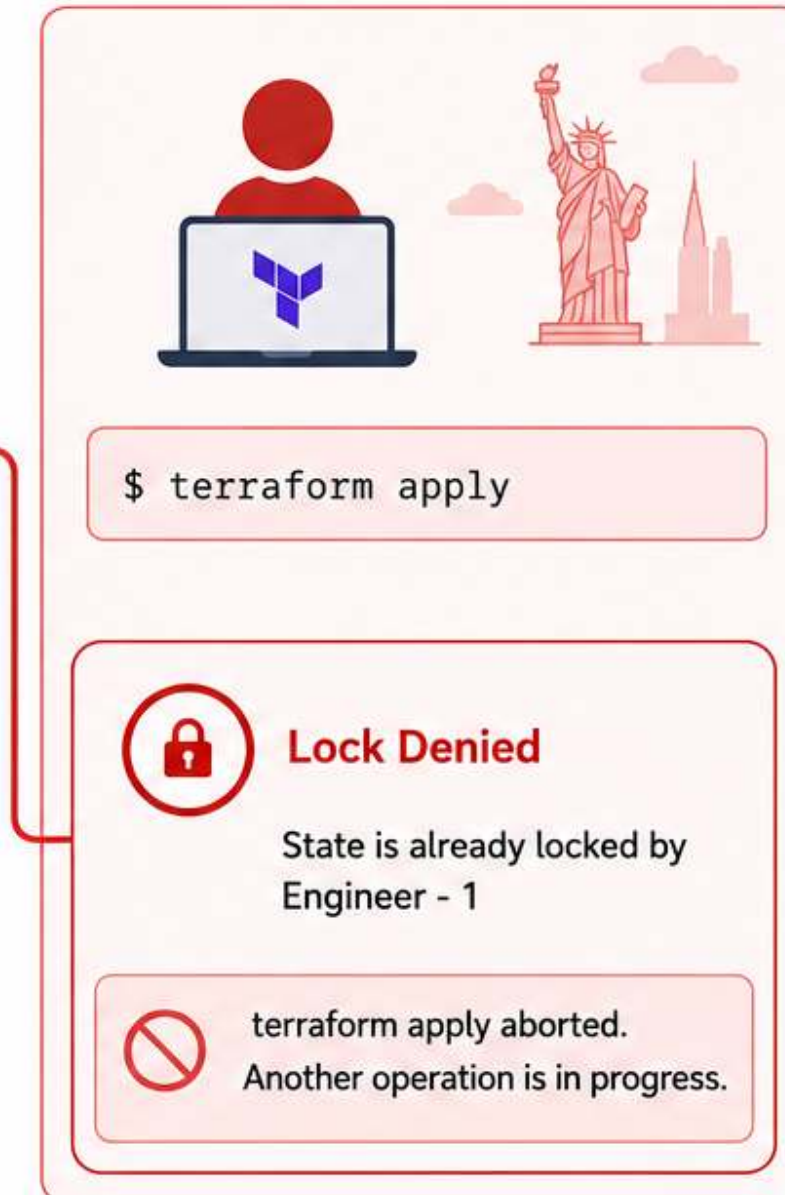
Engineer - 1 (Location: Mumbai)



State Locking Enabled (Prevent concurrent modifications)

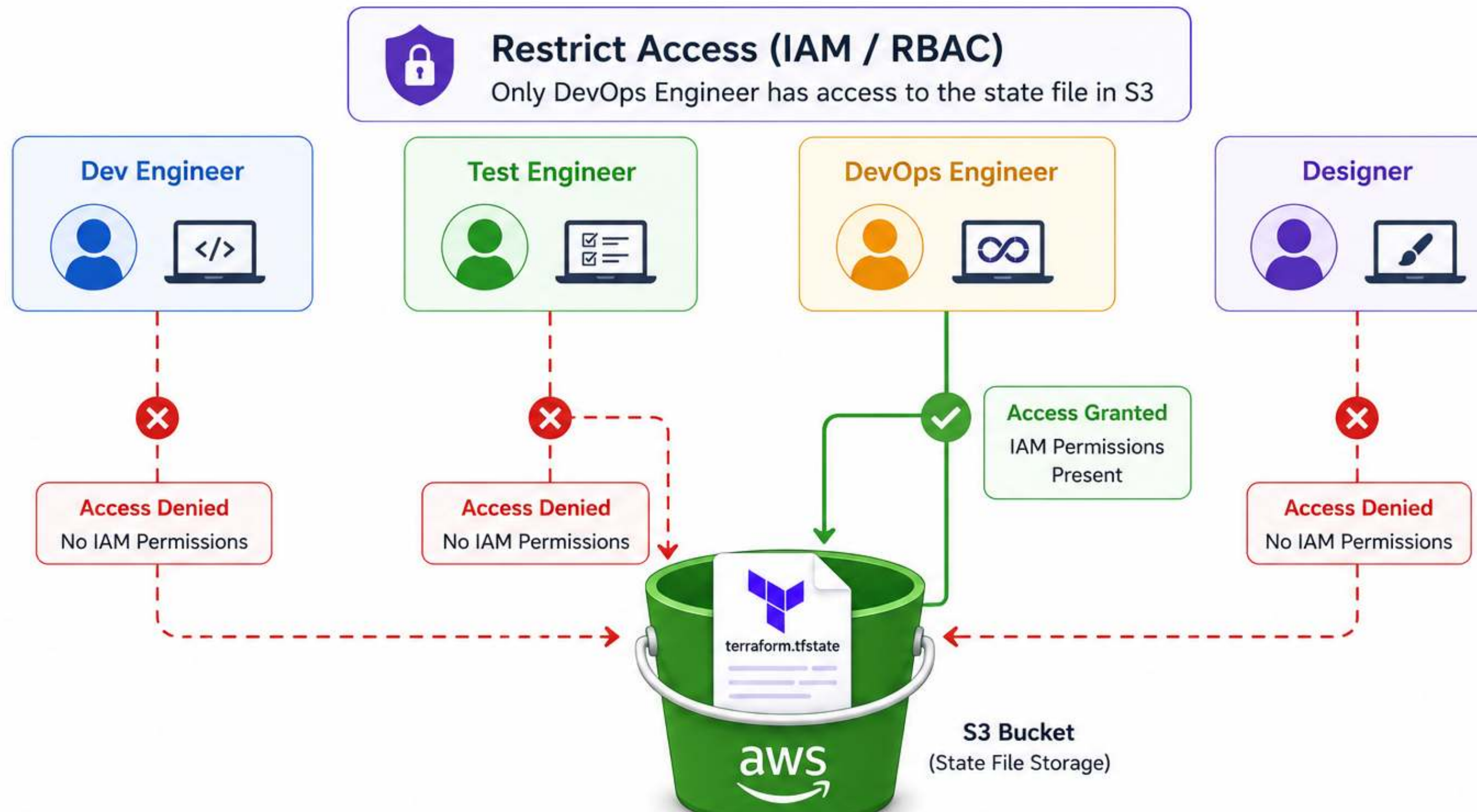


Engineer - 2 (Location: New York)



→ Lock acquired by Engineer - 1

→ Lock request denied for Engineer - 2



Engineer	Role	IAM Permissions	Access to State File
 Dev Engineer	Developer	 No Access	 Access Denied
 Test Engineer	Tester	 No Access	 Access Denied
 DevOps Engineer	DevOps	 Has Access	 Access Granted
 Designer	Designer	 No Access	 Access Denied

IAM Policy
Only DevOps Engineer role has permission to access the S3 bucket



SOME OTHER IMPORTANT POINTS TO SECURE STATE FILE



1. ENABLE BUCKET VERSIONING

It keeps multiple versions of the state file.
Helps in recovery in case of accidental deletion or corruption.



2. DON'T KEEP ANY PASSWORDS ON STATE FILE

Instead, use AWS Secrets Manager or HashiCorp Vault to store secrets securely.



3. NEVER COMMIT YOUR STATE FILE TO GIT

Add it to .gitignore to prevent sensitive data from being exposed.

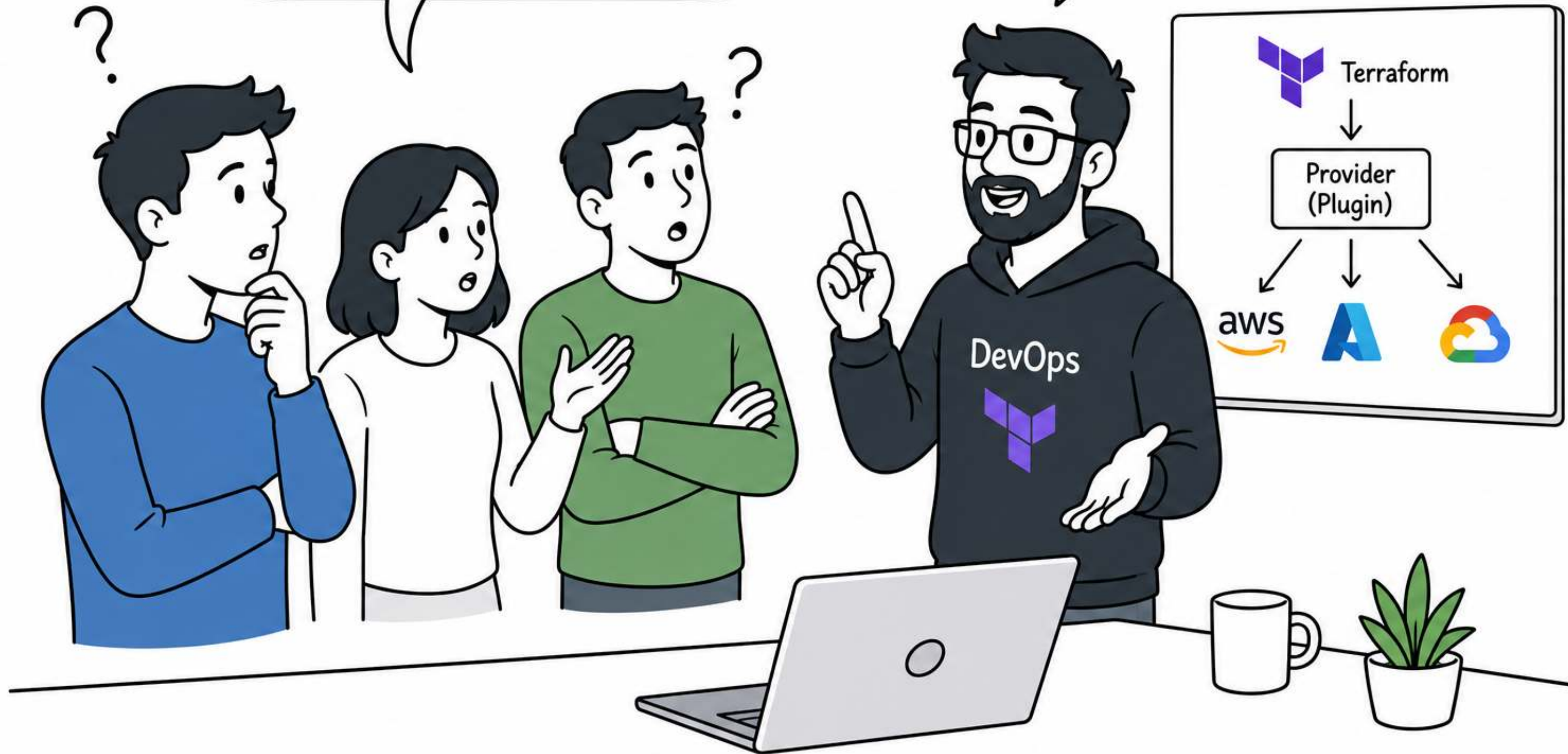
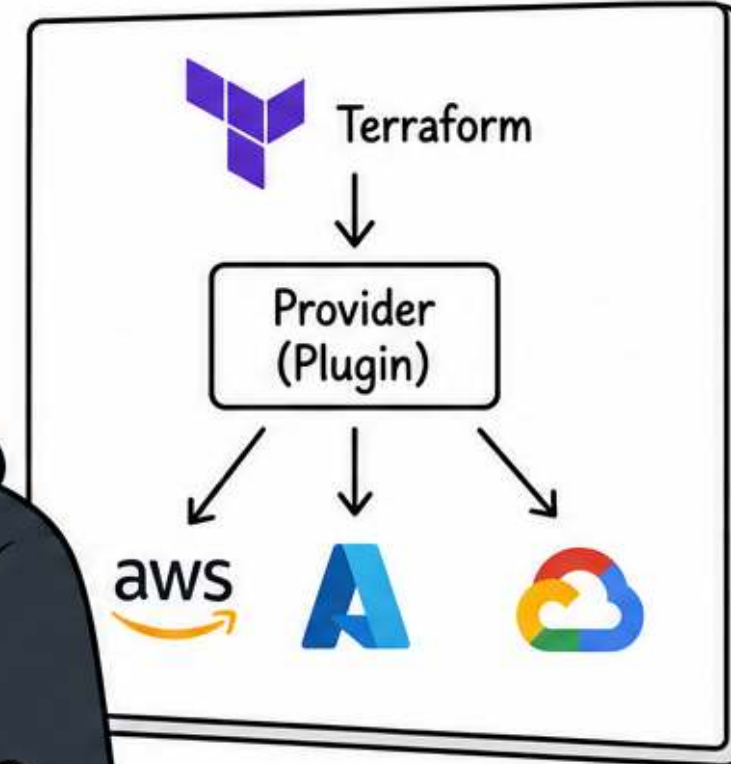
Terraform Code to Create S3 bucket & store statefile

```
provider "aws" {  
  region = "ap-south-1"  
}  
  
resource "aws_s3_bucket" "tf_state" {  
  bucket = "my-company-terraform-state"  
}  
  
resource "aws_s3_bucket_versioning" "versioning" {  
  
  bucket = aws_s3_bucket.tf_state.id  
  versioning_configuration {  
    status = "Enabled"  
  }  
}
```

```
resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {  
  
  bucket = aws_s3_bucket.tf_state.id  
  
  rule {  
    apply_server_side_encryption_by_default {  
      sse_algorithm = "AES256"  
    }  
  }  
}  
  
terraform {  
  
  backend "s3" {  
    bucket = "my-company-terraform-state"  
    key    = "prod/terraform.tfstate"  
    region = "ap-south-1"  
    encrypt = true  
  }  
}
```

How does Terraform know on which provider we are creating resources?

Great question!
Let me explain.

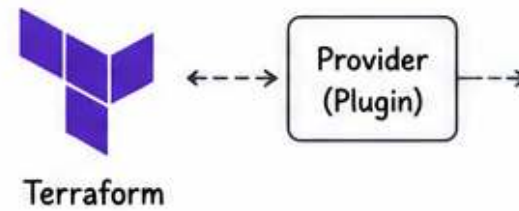


TERRAFORM PROVIDERS

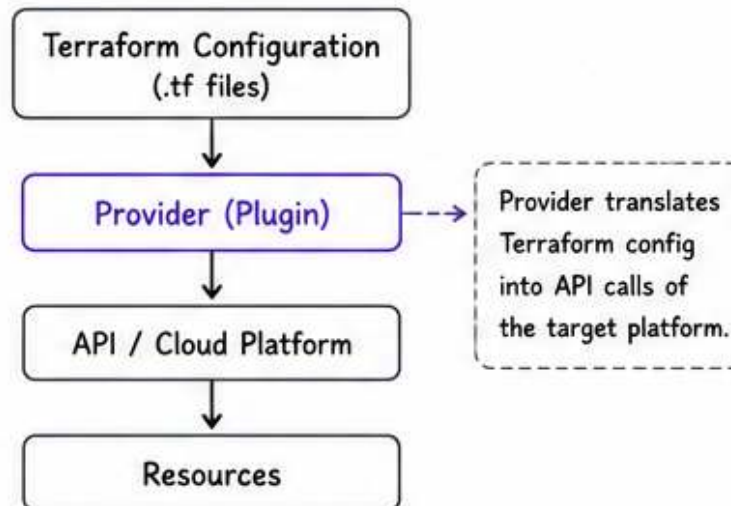
Terraform uses **PROVIDERS** (Plugins) to interact with cloud platforms and other APIs to create, update and manage resources.

1. WHAT IS A PROVIDER?

A provider is a plugin that allows Terraform to communicate with a specific platform or service.



2. HOW IT WORKS



3. EXAMPLE PROVIDERS

-  **AWS Provider**
Manage AWS resources
-  **Azure Provider**
Manage Azure resources
-  **Google Provider**
Manage GCP resources
-  **HashiCorp Provider**
Manage HashiCorp tools like Vault, Consul, Nomad

4. EXAMPLE IN CONFIGURATION

```
provider "aws" {  
  region = "us-east-1"  
}
```

→ This tells Terraform to use AWS Provider to create resources in AWS.

5. KEY TAKEAWAYS

- ✓ Providers are plugins.
- ✓ They act as a bridge between Terraform and the platform.
- ✓ We configure the provider, and Terraform uses it to manage resources.

So, Terraform knows where to create resources because we configure the provider!



```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "6.33.0"  
    }  
  }  
}
```

```
provider "aws" {  
  # Configuration options  
}
```

TERRAFORM WORKSPACE:

Use the same Terraform code but maintain multiple separate state files.

Lets assume that i have 3 identical environments, instead of writing the code for 3 environments we can simply use one code for 3 environments.

Ex: i have 3 services in feature branch and i would like to perform testing for these 3 features. In this case i need 3 infra's with same configurations with different statefiles. Here we can use terraform workspaces

COMMANDS FOR WORKSPACE:

COMMAND	PURPOSE
terraform workspace show	show active workspace
terraform workspace list	list workspaces
terraform workspace new NAME	create workspace
terraform workspace select NAME	switch workspace
terraform workspace delete NAME	delete workspace
terraform.workspace	use workspace inside code

```

provider "aws" {
  region = var.region
  access_key = ""
  secret_key = ""
}

resource "aws_instance" "example" {
  ami          = var.ami
  instance_type = var.size
  tags = {
    Name = "demo-server"
  }
}

variable "region" {
  default = "us-east-1"
}

variable "ami" {
  default = "ami-00b8917ae86a424c9"
}

variable "size" {
  default = "t2.micro"
}

```

- terraform init
- terraform plan
- terraform apply

This will create instance in default workspace

Create new workspace called dev

- terraform init
- terraform plan
- terraform apply

This will create one more instance with same config on **dev env**

Even though there is one instance already up & running on AWS, terraform will create one more instance because we run the terraform commands from different workspace!

Create new workspace called prod

- terraform init
- terraform plan
- terraform apply

This will create one more instance with same config on **prod env**

As per the above code, all the instances will get same instances names, if you wish to change it then change the instance name like below code:

```
provider "aws" {  
  region = var.region  
}  
  
locals {  
  instance_types = {  
    dev    = "t2.micro"  
    test   = "t2.small"  
    prod   = "m4.large"  
  }  
}  
  
resource "aws_instance" "demo_server" {  
  ami          = var.ami  
  instance_type = local.instance_types[terraform.workspace]  
  tags = {  
    Name = "example-server-${terraform.workspace}"  
  }  
}
```

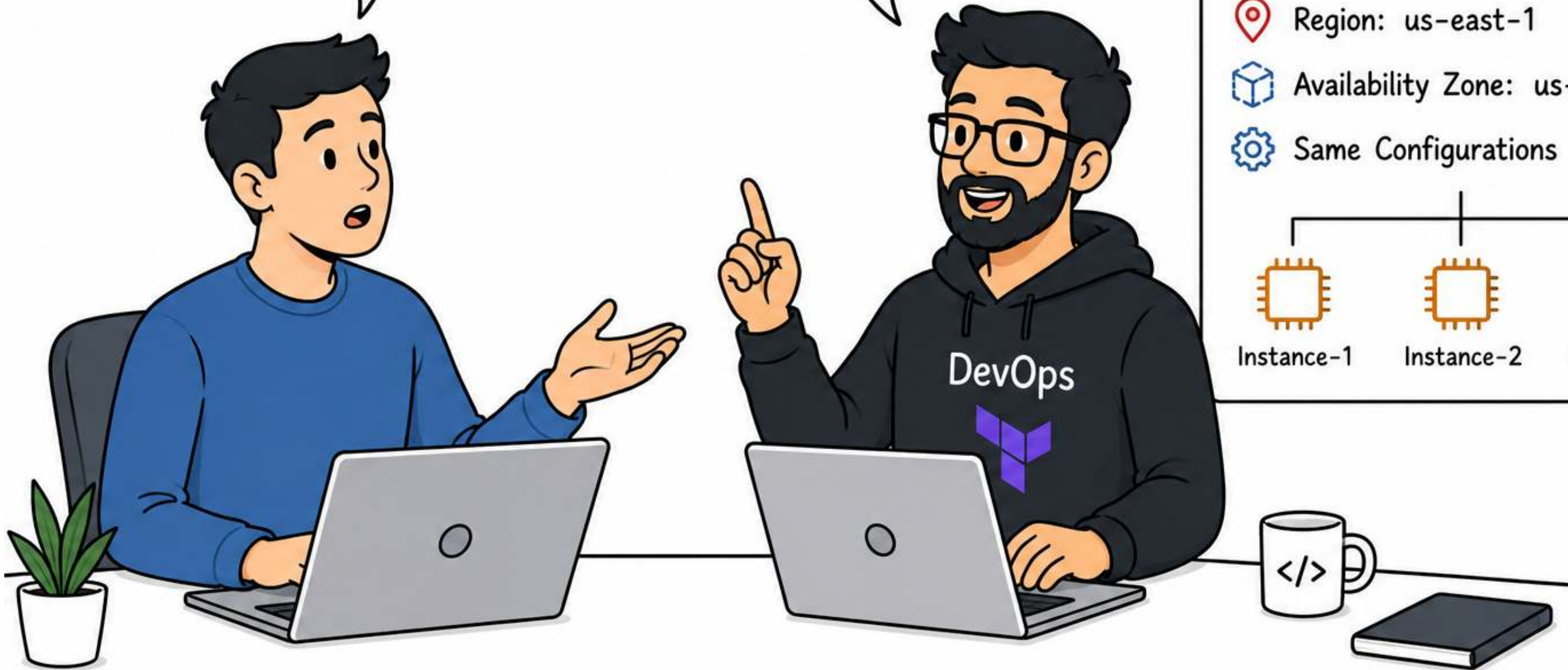
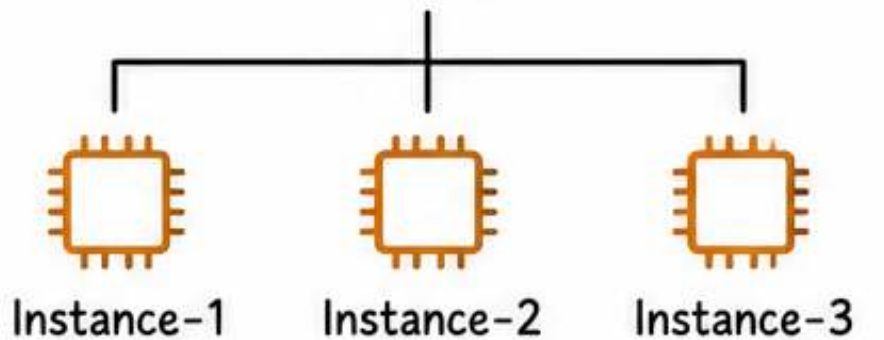
If we execute the above code, that will create different sizes for instances with different names

What's the task for me?

Create 3 instances on us-east-1a with the same configurations.

TASK DETAILS

- ☒ Create 3 Instances
-  Region: us-east-1
-  Availability Zone: us-east-1a
-  Same Configurations



Okay! I'll create
3 main.tf files to
create 3 instances.

main-1.tf

```
resource "aws_instance" "web1" {  
  ami           = "ami-0abcd1234"  
  instance_type = "t2.micro"  
  availability_zone = "us-east-1a"  
  # ... same configurations  
}
```

main-2.tf

```
resource "aws_instance" "web2" {  
  ami           = "ami-0abcd1234"  
  instance_type = "t2.micro"  
  availability_zone = "us-east-1a"  
  # ... same configurations  
}
```

main-3.tf

```
resource "aws_instance" "web3" {  
  ami           = "ami-0abcd1234"  
  instance_type = "t2.micro"  
  availability_zone = "us-east-1a"  
  # ... same configurations  
}
```

Haha! There's a
better way to do this!
Let me show you.

DevOps



Terraform

So we don't need
3 files?
Only 1 file is enough
to create 3 instances?

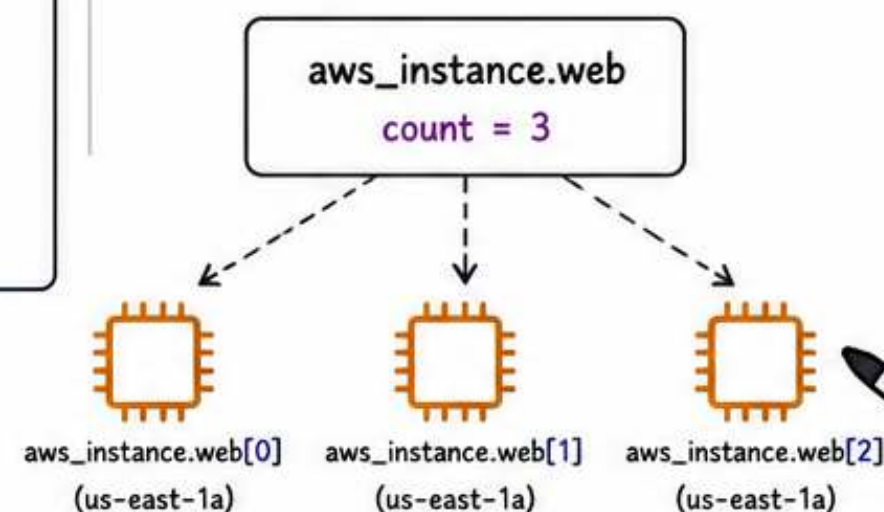
1. SINGLE FILE - main.tf

```
resource "aws_instance" "web" {  
  count = 3  
  ami           = "ami-0abcd1234"  
  instance_type = "t2.micro"  
  availability_zone = "us-east-1a"  
  # ... same configurations  
}
```

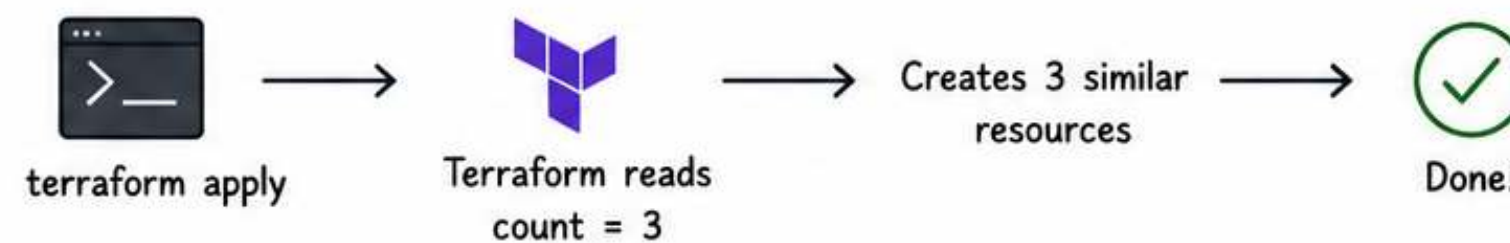
Yes! Just 1 file is enough.
We use 'count' meta-argument.

2. HOW COUNT WORKS

- count = 3 tells Terraform to create 3 instances of this resource.
- Terraform creates indexed instances starting from 0.



3. WHAT HAPPENS WHEN YOU RUN?



4. BENEFITS

- ✓ Less code (DRY)
- ✓ Easy to scale (just change count value)
- ✓ Consistent configurations



Example:

Change `count = 3` to `count = 5`
→ It will create 5 instances!





main.tf

```
resource "aws_s3_bucket" "bucket" {  
  bucket = "Mustafa.77.bucket"  
  count  = 2  
}
```

```
>_ $ terraform apply
```

Error: creating S3 Bucket (Mustafa.77.bucket):
BucketAlreadyExists: The requested bucket name
is not available.
status code: 409, request id: XXXX, host id: YYYY

with aws_s3_bucket.bucket[1],
on main.tf line 1, in resource "aws_s3_bucket" "bucket":
1: resource "aws_s3_bucket" "bucket" {

What happened?

- You used the same bucket name for multiple resources.
- S3 bucket names must be globally unique.

Solution?

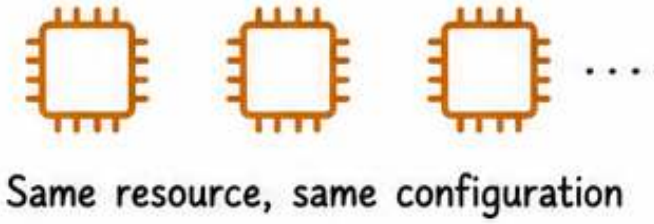
- Use count.index to make each bucket name unique.



I used count but got an error. When should I use count vs for_each?

1 What is count?

- count helps you create multiple instances of the SAME resource with similar configurations.



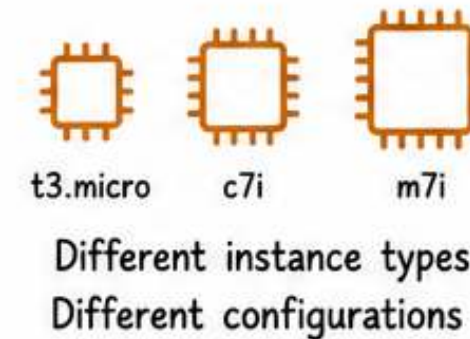
2 Important Note

- Some resources need UNIQUE names (e.g., S3 buckets).



3 When to use for_each?

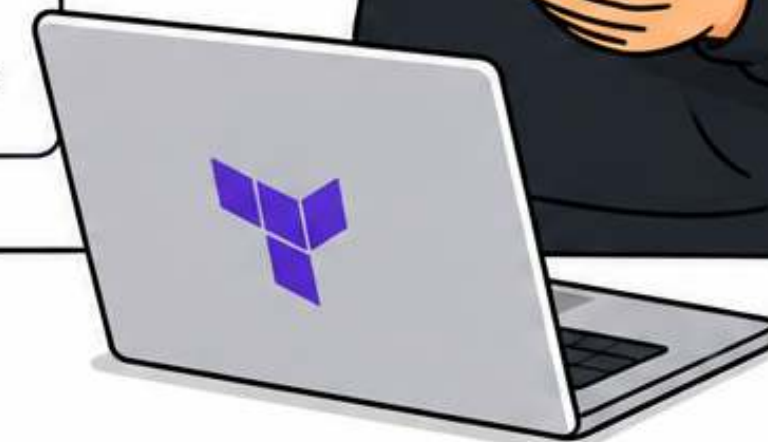
- Use for_each when:
 - ✓ You need to create multiple resources with DIFFERENT configurations.
 - ✓ You have a list or map of values.



Key Takeaway

- count → for same resources, similar configuration
- for_each → for different configurations or when uniqueness matters

Let me explain the difference in simple points.



Terraform Code to Create S3 buckets using **for_each**

```
provider "aws" {  
  region = "ap-south-1"  
}  
  
resource "aws_s3_bucket" "bucket" {  
  
  for_each = toset([  
    "mustafashaik77.dev-bucket",  
    "mustafashaik77.test-bucket",  
    "mustafashaik77.uat-bucket"  
  ])  
  
  bucket = each.key  
  
}
```

```
variable "instances" {  
  type = map(object({  
    instance_type = string  
    volume_size   = number  
    env           = string  
  }))  
  
  default = {  
    app = {  
      instance_type = "t3.micro"  
      volume_size   = 20  
      env           = "dev"  
    }  
  
    api = {  
      instance_type = "t3.small"  
      volume_size   = 50  
      env           = "staging"  
    }  
  
    db = {  
      instance_type = "t3.large"  
      volume_size   = 100  
      env           = "prod"  
    }  
  }  
}
```

Terraform Code to Create EC2 Instances using **for_each**

```
variable "instances" {
  type = map(object({
    instance_type = string
    volume_size   = number
    env           = string
  }))

  default = {
    app = {
      instance_type = "t3.micro"
      volume_size   = 20
      env           = "dev"
    }

    api = {
      instance_type = "t3.small"
      volume_size   = 50
      env           = "staging"
    }

    db = {
      instance_type = "t3.large"
      volume_size   = 100
      env           = "prod"
    }
  }
}
```

```
resource "aws_instance" "server" {
  for_each = var.instances

  ami           = "ami-xxxxxx"
  instance_type = each.value.instance_type

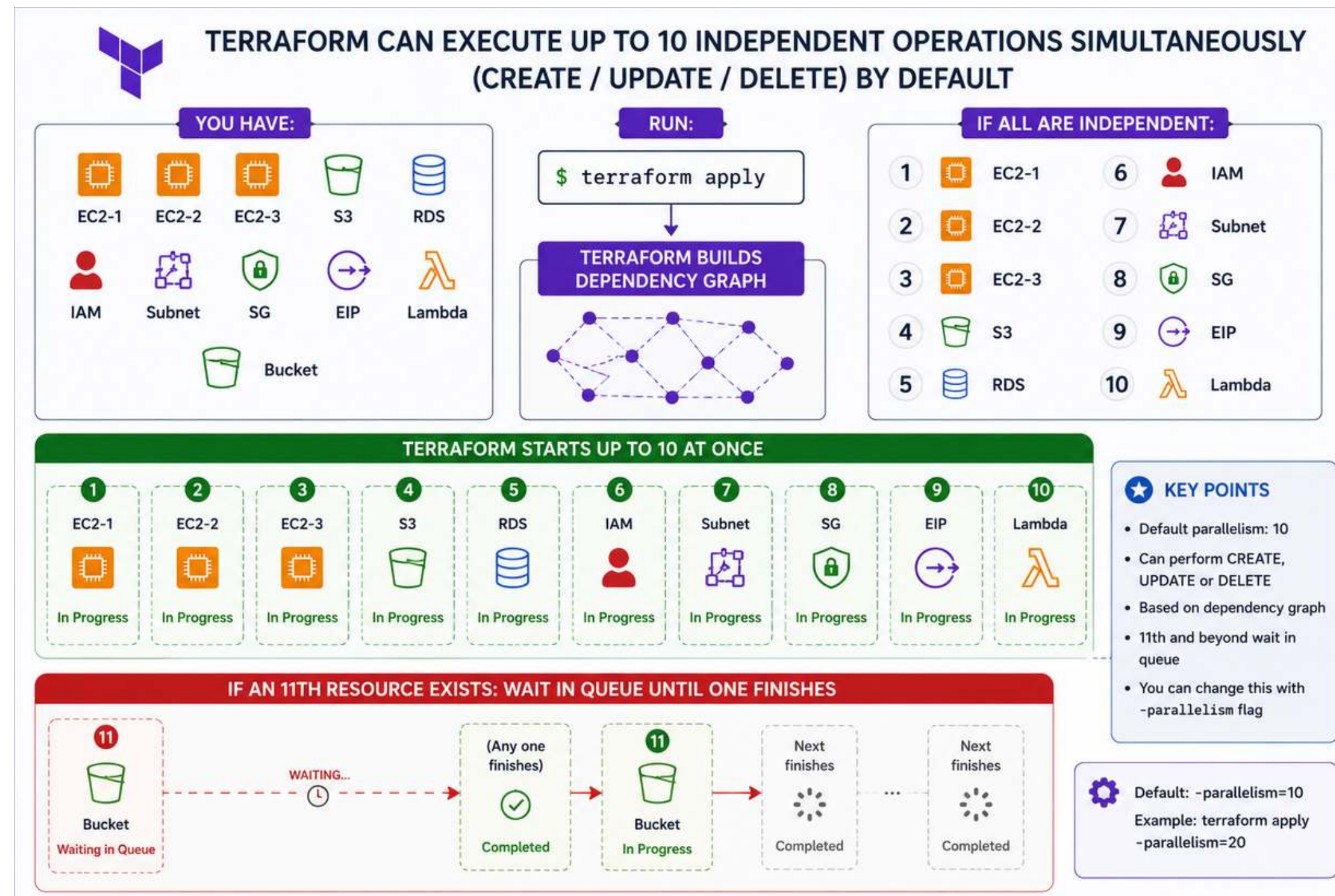
  root_block_device {
    volume_size = each.value.volume_size
  }

  tags = {
    Name = each.key
    Env  = each.value.env
  }
}
```



TERRAFORM PARALLELISM

Terraform creates, updates, or destroys multiple independent resources at the same time instead of one by one.



Can we
increase the limit?

Of course!
We can increase/decrease
the parallelism limit using:



Increase Parallelism

```
terraform apply -parallelism=20
```

(or)



Decrease Parallelism

```
terraform apply -parallelism=2
```


DevOps

What is
`depends_on`
in Terraform?

`depends_on` is used to tell Terraform:
“Wait... finish creating this
resource **first**, then create
the **next** one.”

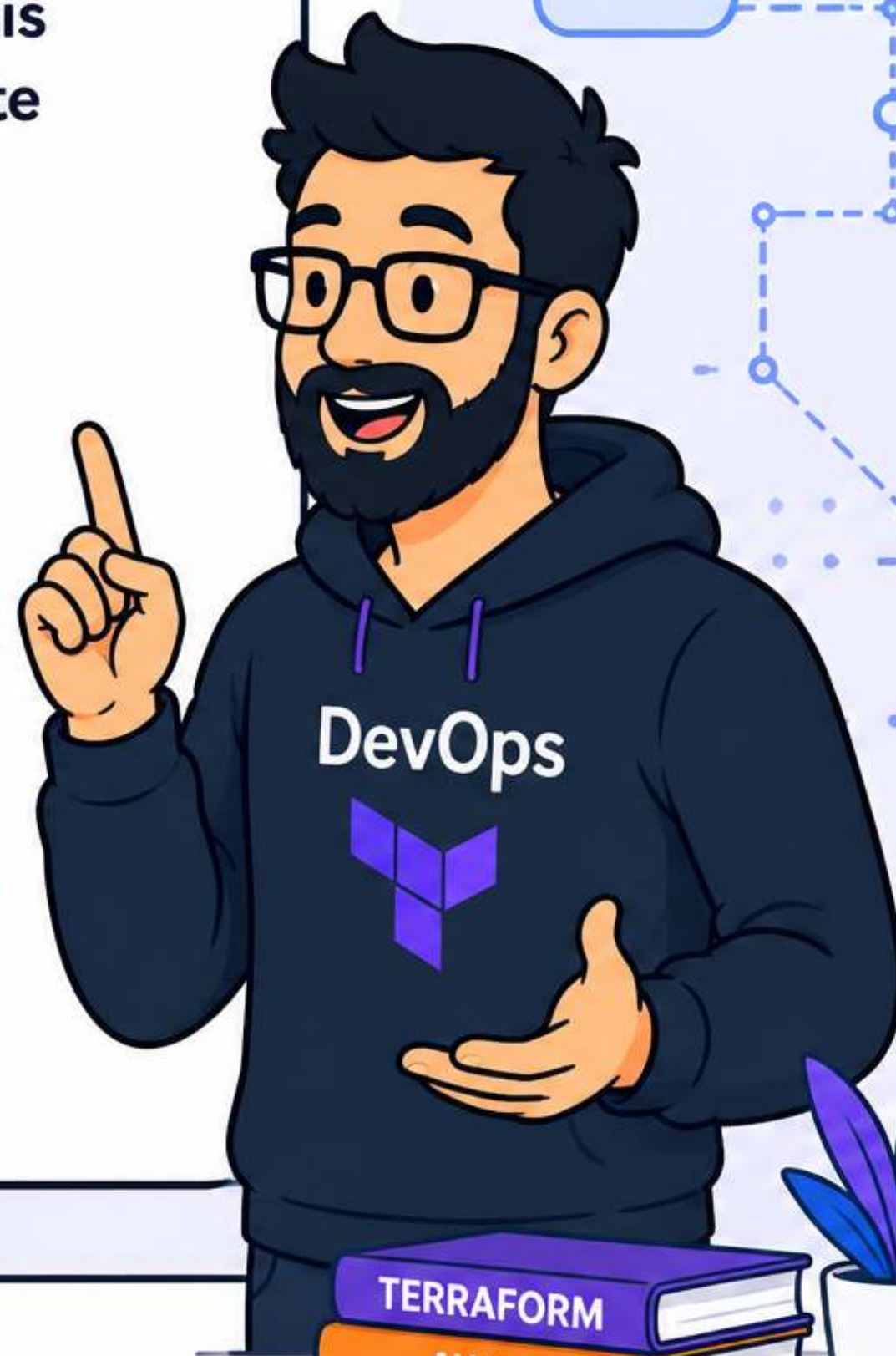
Example:



Create **VPC** first



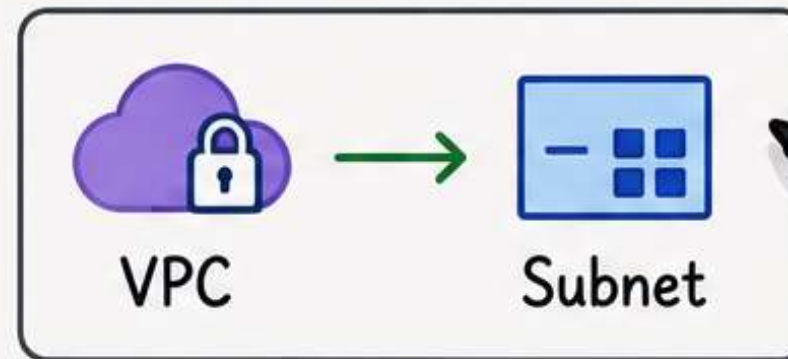
Then create **EC2**



depends_on in Terraform

- ✓ Usually, Terraform detects dependencies **automatically**.

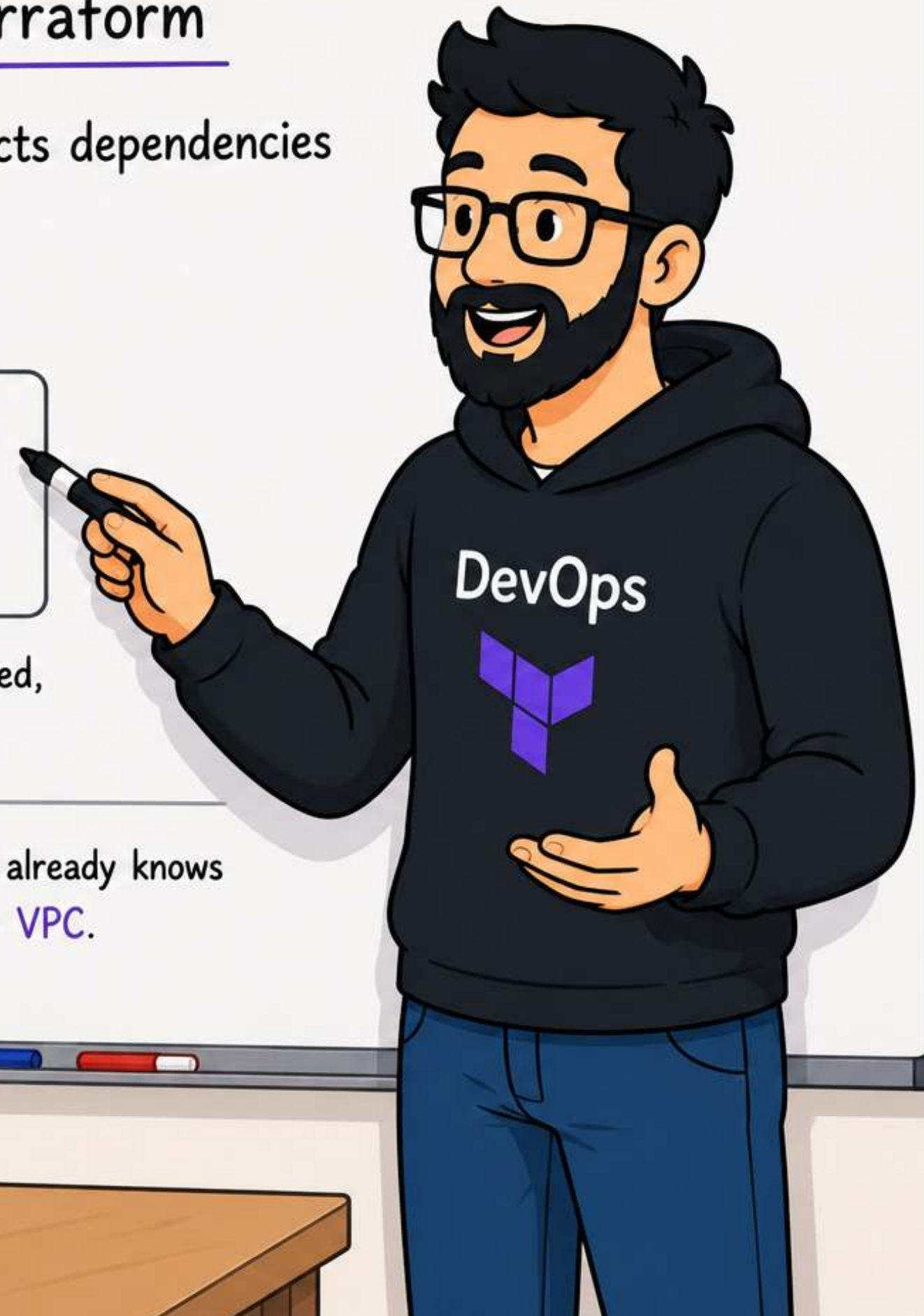
- Example:



First **VPC** needs to be created, and then **Subnet**.



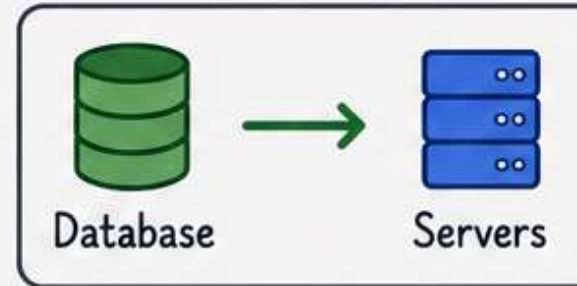
Why? Because Terraform already knows that **Subnet** depends on **VPC**.



Sometimes, we need to use `depends_on` for some resources as per our requirement.

① Application Dependency

Database needs to create first, then Servers.



② IAM Propagation

IAM Role needs to create first, then EC2 Instance.



⚠ NOTE:

Don't use unnecessary `depends_on` because it may slow down the process of creating the infra.



Use `depends_on` only when Terraform can't automatically understand the dependency.



